



数字逻辑与处理器基础

Fundamentals of Digital Logic and Processor



第八讲 存储器与总线外设

第2-1部分 Cache性能评估

李学清

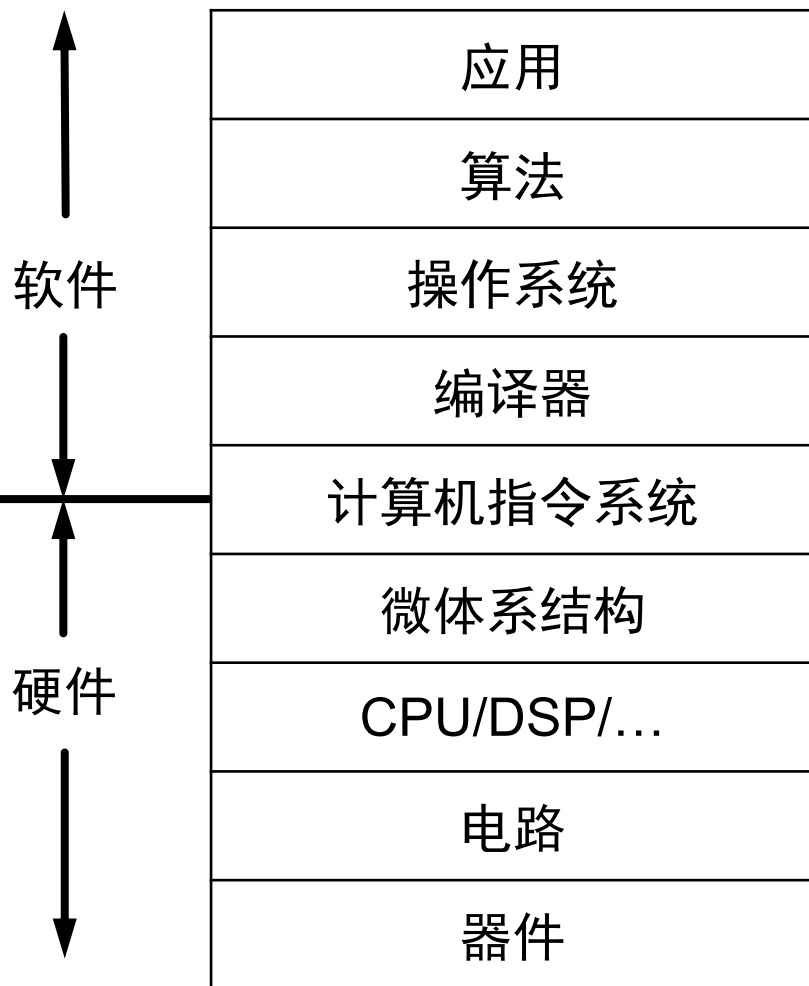
#腾讯会议： 453-1488-6548

助教：唐文骏*/陈仲豪/曾令安

*本节内容负责答疑助教

2024年12月23日

微体系结构



计算机系统

周	日期	暂定主要内容
1	9/9	绪论
2	9/21	布尔代数 (中秋节调休)
3	9/23	组合逻辑
4	9/30	时序逻辑
5	10/7	国庆节放假
6	10/14	时序逻辑
7	10/21	计算机指令系统
8	11/3	期中考试
9	11/4	计算机指令系统
10	11/11	微处理器
11	11/18	微处理器
12	11/25	流水线
13	12/2	流水线
14	12/9	流水线
15	12/16	存储器
16	12/23	存储器/IO与总线/总结

目录

- 复习与回顾
- 存储器系统基础
- 存储器系统的设计理论
- 缓存技术
- 缓存性能评估

缓存技术实现

- **Cache技术实现**

1. **Cache基本结构**

- Cache的存储体结构、寻址流程

2. **Cache地址映像方式**

- 直接映像、全相连映像、组相联映像

3. **Cache数据的替换及更新**

4. **Cache的作用**

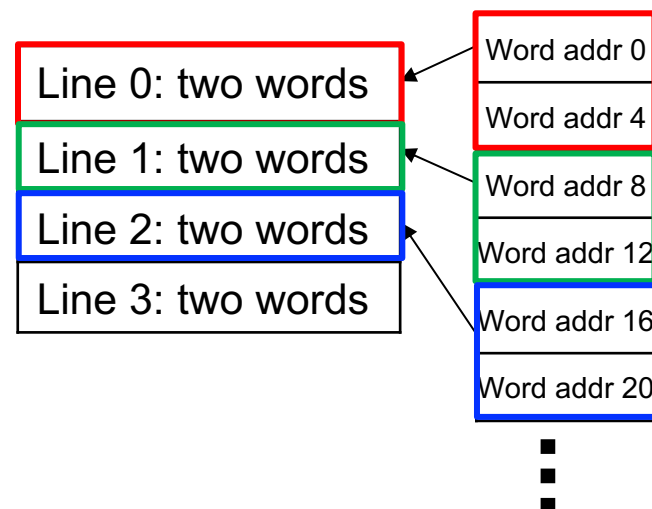
缓存技术：Cache的作用

- 如下代码和对应指令，\$1和\$4初始值均为0，假定指令地址从0开始

```
loop:  lw  $2, 1000($1)
        addi $1, $1, 4
        add  $4, $2, $4
        bne $1, 1000, loop
```

```
int sum = 0;
for(int i=0; i<1000/4; i++)
    sum = sum + a[i];
```

- 假定我们有如下的cache
 - 直接映像
 - Cache总容量八个字(32 bytes)
 - Cache共4行，每行两个字(8 bytes)
- 若指令和数据a[]分别有cache，上面代码的平均命中率为多少？



缓存技术：Cache的作用

- 如下代码和对应指令，\$1和\$4初始值均为0，假定指令地址从0开始

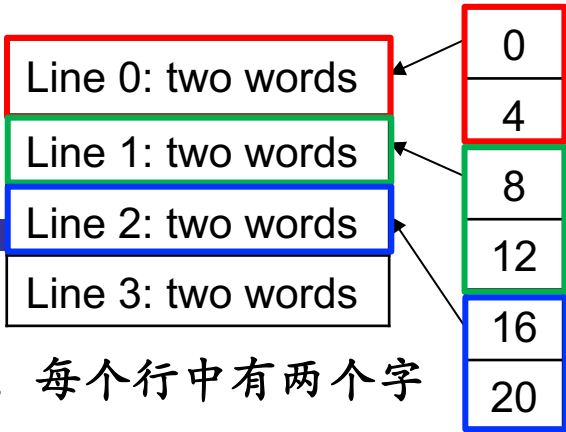
```
loop:  lw  $2, 1000($1)
        addi $1, $1, 4
        add  $4, $2, $4
        bne  $1, 1000, loop
```

```
int sum = 0;
for(int i=0; i<1000/4; i++)
    sum = sum + a[i];
```

- **Step1：确定数据和指令的“轨迹”**
- 指令地址序列(考虑流水线结构导致的stall):
 - 0,4,8,12,16, 0,4,8,12,16, ..., 0,4,8,12,16 (十进制数)
- 数据地址序列:
 - 1000,1000+4,1000+8,...,1000+(1000-4)
 - 1000,1004,1008,...,1996

缓存技术：Cache的作用

- Step2: 指令过程分析



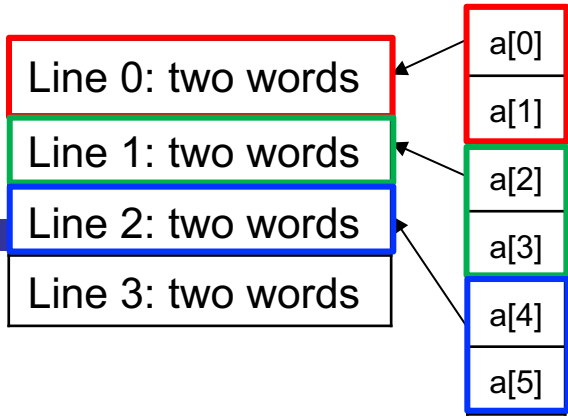
直接映像，Cache共4行，每个行中有两个字

访问次数	1	2	3	4	5	6	7	8	9	10
指令标号	0	4	8	12	16	0	4	8	12	16
地址块号	0	0	1	1	2	0	0	1	1	2
行分配	0	0,4	0,4	0,4	0,4	0,4	0,4	0,4	0,4	0,4
	1	-	-	8,12	8,12	8,12	8,12	8,12	8,12	8,12
	2	-	-	-	-	16,20	16,20	16,20	16,20	16,20
	3	-	-	-	-	-	-	-	-	-
操作状态		调进		调进		调进				
是否命中		未命中	命中	未命中	命中	未命中	命中	命中	命中	命中
		data (inst.) not yet in cache				已经在cache里，直接访问				

指令缺失率 = $3 / (5 * 250) = 0.24\%$ ，指令命中率 = $1 - 0.24\% = 99.76\%$

缓存技术：Cache的作用

- Step2: 数据过程分析



1000 → 0b0011 1110 1000

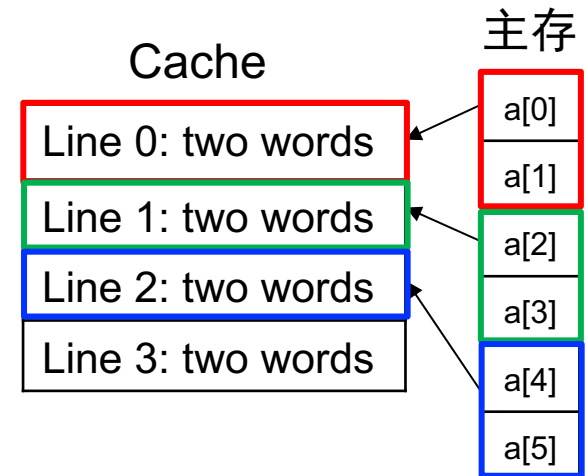
行号 行内地址

访问顺序	1	2	3	4	5	6	7	8	9	10
地址块号	125	125	126	126	127	127	128	128	129	129
行分配情况	00	00	00	00	00	00	01	01	10	01
	01	01	01	01	01	01	01	01	01	01
	10	10	10	10	10	10	10	10	10	10
	11	11	11	11	11	11	11	11	11	11
操作状态	调进		调进		调进		调进		替换	
是否命中	未命中	命中	未命中	命中	未命中	命中	未命中	命中	未命中	命中

数据缺失率 = 数据命中率 = 50%

缓存技术：Cache的作用

- 假定我们有如下一个Cache
 - Cache总容量八个字(32 bytes)
 - 每个cache行中有两个字(8 bytes)
- 有如下的地址轨迹
 - 1000,1032,1000,1032,...
- 请思考
 - 直接映像的性能如何？
 - 采用LRU替换策略的2路组相联映像性能如何？和直接映像有差别吗？



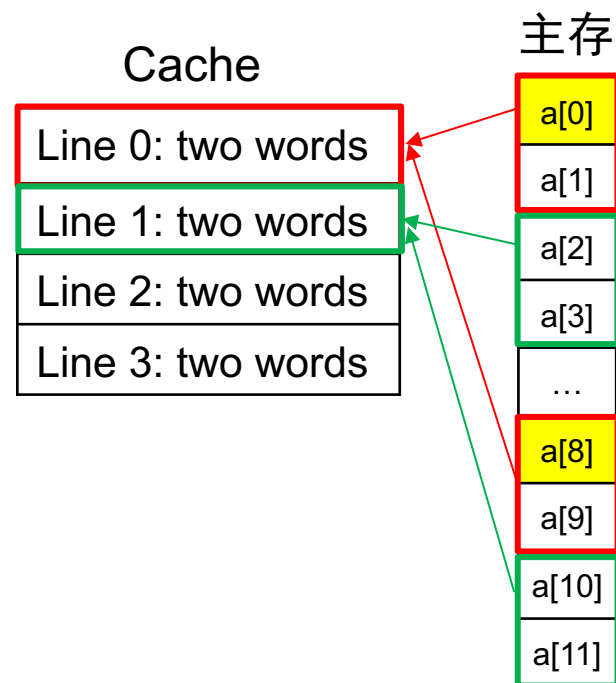
缓存技术：Cache的作用

- 直接映像

1000 → 0b0011 1110 1000
1032 → 0b0100 0000 1000

行号 行内地址

访问顺序	1	2	3	4
地址块号	125	129	125	129
行分配情况	- 125 - -	- 129 - -	- 125 - -	- 129 - -
操作状态	调进	替换	替换	替换
是否命中	未命中	未命中	未命中	未命中



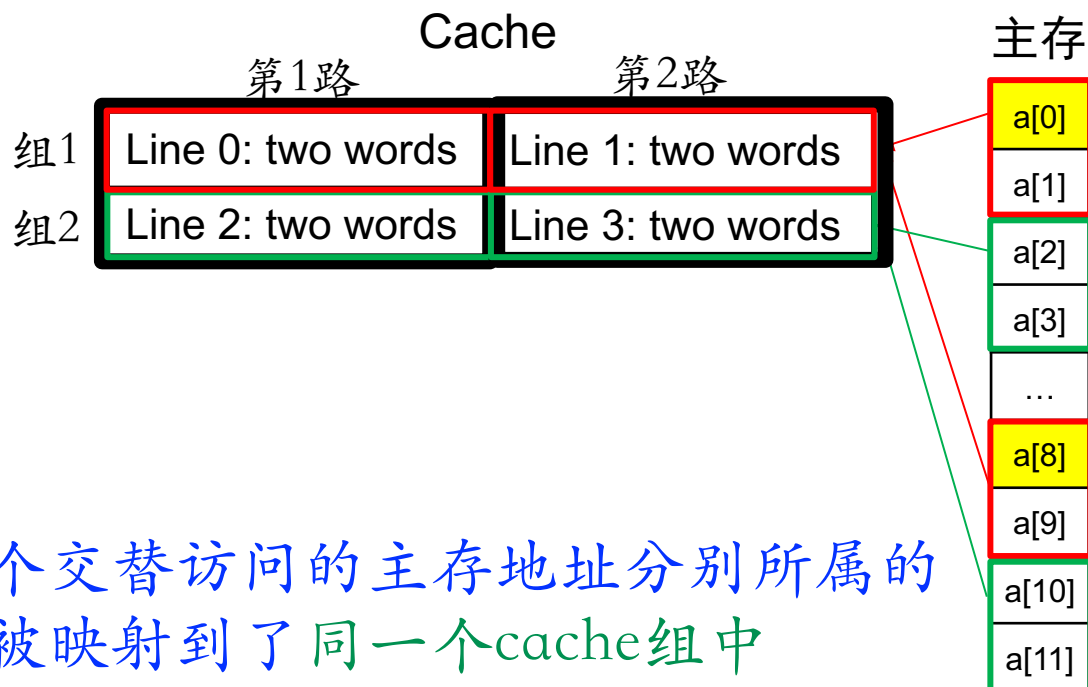
- 两个交替访问的主存地址分别所属的块被映射到了同一个cache行中，因此每次访问都发生缺失，缺失率 = 100%，命中率 = 0%

缓存技术：Cache的作用

- 2路组相联映像

1000 → 0b0011 1110 1000
1032 → 0b0100 0000 1000

标签 组号 行内地址



a[0] 和 a[8] 可以同时
存在Cache中 (组1)

- 两个交替访问的主存地址分别所属的块被映射到了同一个cache组中
 - 行0和行1
 - 块(a[0]&a[1]) 和块(a[8]&a[9])

缓存技术：Cache的作用

• 2路组相联映像

访问顺序

1

2

3

4

组号

地址块号

125

129

125

129

行分配情况

-	-
125	-

-	-
125	129

-	-
125	129

-	-
125	129

操作状态

调进

调进

命中

命中

是否命中

未命中

未命中

1000



0b0011 1110



1032



0b0100 0000

数据缺失仅发生在前两次访问

目录

- 复习与回顾
- 存储器系统基础
- 存储器系统的设计理论
- 缓存技术
- 缓存性能评估

缓存性能评估

- 缓存性能评估
 1. Cache缺失的3C原则
 2. Cache评价指标
 3. 如何提高Cache性能

缓存性能评估：3C原则

- CPU的性能方程：

- CPU执行时间=指令数×CPI×时钟周期
- 在存储器的角度来看，存储器的访问延时会带来CPI的增加。取恒定的存储器阻塞周期，存在如下关系：

$$\text{实际CPI} = \text{理想CPI} + \text{存储器阻塞周期数}$$

- 实际CPI受限于指令和数据的AMAT(Average Memory Access Time):

$$\text{AMAT} = \text{命中访问时间} + \text{缺失率} \times \text{缺失时间代价}$$

- 目标：研究缺失的原因的解决方法，降低AMAT

缓存性能评估：3C原则

- 3C原则

- 强迫性缺失(Compulsory): 第一次访问主存储器中的某一个数据块, 只能先从主存储器将数据加载到cache中, 也称为冷启动缺失或首次访问失效
- 容量缺失(Capacity): 由于cache容纳不了程序所需的所有主存块而引起的缺失。
- 冲突缺失(Conflict): 在组相联或者直接映像中, 多个的主存块竞争同一个cache组时引起的缺失且该存储块曾经存在于cache中, 也称碰撞缺失（前提: cache未满）。

注意: 所有新的数据块第一次进入cache时都为强迫性缺失, 替换出去后再次进入cache按情况分为容量缺失或冲突缺失。

缓存性能评估：3C原则

- 容量 vs 冲突

- 容量缺失

- 当缓存中的**所有**行都已被使用，但是有新的块要放进缓存中（缓存若不替换掉一行就存不下这个新的块）

- 冲突缺失（碰撞缺失）

- 当缓存中还有空的行时，若有一个新的块要放进缓存中的某一组，而该组的所有行都已经用了（存了别的块），这个新的块需要替换掉其中的某一行
 - 只出现在**组相联**和**直接映射**的缓存中出现

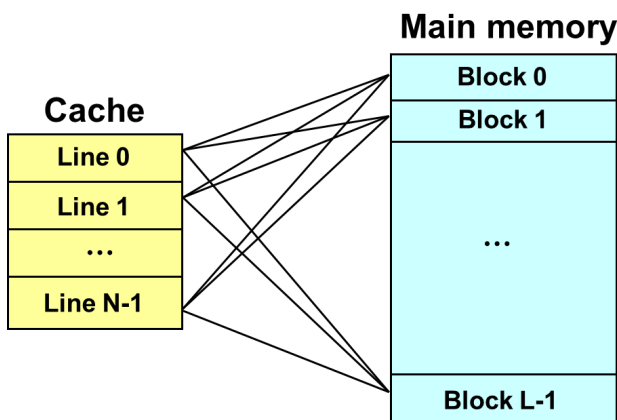
类比课上例子：教室还有空位，却“非要挤一个同学的座位”，这是冲突缺失；只有教室没有空位了，“挤他人座位”才是容量缺失。

注意：在此处及课件的后续页中，讨论容量缺失及冲突缺失时都已假设了这些数据块都曾经存在于cache中，即这些块都已发生过一次强迫性缺失。

缓存缺失: 3C原则

• 全相联 Cache

- 强迫性缺失: **存在**
 - 由第一次访问从未在缓存中出现过的区块引起的。也被称为冷启动失误或无效的第一次访问
- 冲突缺失: **不存在**
 - 当块数多于同一组路数, 且在缓存中实际上有**未填满**的行
- 容量缺失: **主导**
 - 当缓存中所有行都已被使用, 不能包含更多需要的块时



	强制	容量	冲突
全相联	存在	主导	/

缓存缺失: 3C原则

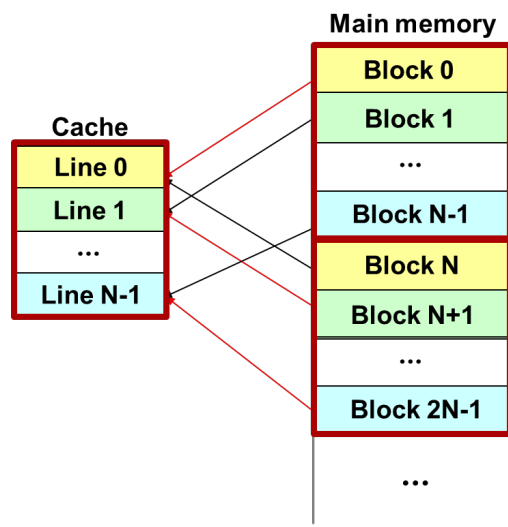
- 直接映射 Cache

- 强迫性缺失: 存在
- 容量缺失: 存在

- 当缓存中的所有行都已被使用, 不能包含更多需要的区块时引起的

- 冲突缺失: 主导类型

- 当缓存中该组的所有行都已被使用, 但是在其他组中实际上还有未用的行

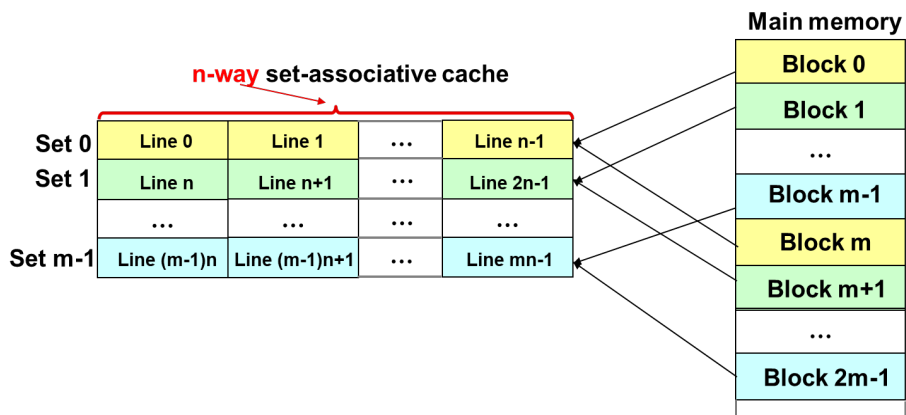


	强制	容量	冲突
全相联	存在	主导	/
直接映射	存在	存在	主导

缓存缺失: 3C原则

• 组相联 Cache

- 强迫性缺失: 存在
- 冲突缺失: 主导
 - 当块的数量多于同一组中路的数量, 并且在其他组中实际上有未填满的行
- 容量缺失: 存在
 - 当缓存中的所有行都已被使用, 不能包含更多需要的区块时引起的



	强制	容量	冲突
直接映射	存在	存在	主导
全相联	存在	主导	/
组相联	存在	存在	主导

缓存缺失: 3C原则

- **3C 原则 和 Cache 映射**

- 强迫性缺失在所有Cache映射方式中都存在。
- 冲突缺失在**全相联**Cache中**不存在**
 - 如果所有行都满了，是容量不足。
- 在直接映射和组相联Cache中，容量缺失比较少见（所有行都填满的可能性较小）
 - 如果一个Cache行需要被替换
 - 所有其他行均被填满: 容量缺失
 - 有一些行没被填满: 冲突缺失

	强制	容量	冲突
直接映射	存在	存在	主导
全相联	存在	主导	/
组相联	存在	存在	主导

缓存性能评估：3C原则

- 以常见的循环操作为例

Q. 无限容量全相联Cache中只会发生哪一种缺失？

A. 强迫性缺失。在首次访问主存块时必定发生

Q. 有限容量全相联Cache中哪一种缺失占主导？

A. 容量缺失。Cache中没有的更多的行存储数据

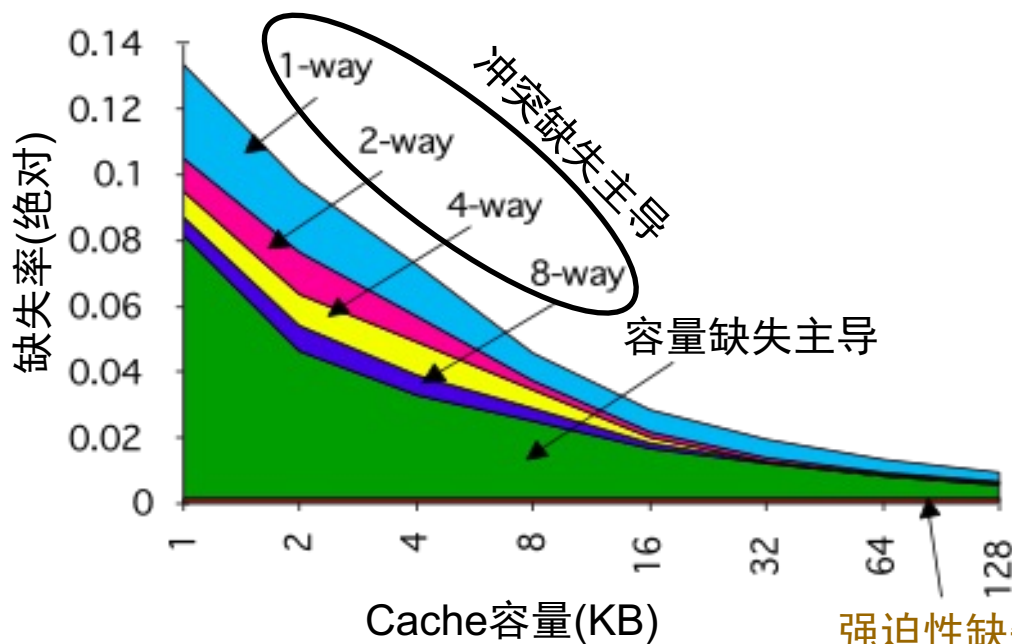
Q. 组相联或直接映像Cache中哪一种缺失占主导？

A. 冲突缺失。冲突相当于组内(局部)容量缺失，其发生的概率大于全局的容量缺失。

缓存性能评估：3C原则

- 3C原则的统计规律(1)

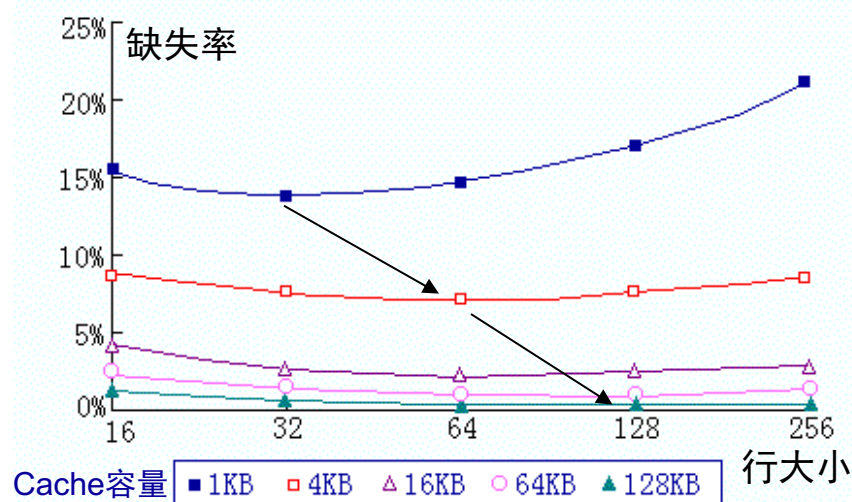
- 强迫性缺失和cache容量大小无关（和块的大小有关）
- 容量缺失随着cache容量增大而减少
- 冲突缺失随着相联度增大而减少



缓存性能评估：3C原则

- 3C原则的统计规律(2)

- 对于给定的cache容量，随着cache行大小的增加，缺失率会先下降后上升。因为强迫性缺失会下降，但是容量缺失和冲突缺失会增加。
- Cache的容量越大，使得缺失率达到最低点的cache行大小也越大



缓存性能评估：3C原则

- 减少强迫性缺失

- 解决方案：增大cache行的大小（即：块的大小）
- 优点：
 - 能够有效减少强迫性缺失，尤其对指令cache非常有效
- 缺点：
 - 会造成更多的冲突性缺失
 - 可能会读入很多没有必要数据

	强制缺失
Cache容量↑	-
Cache行大小↑	↓
Cache相联度↑	-

缓存性能评估：3C原则

- 减少容量缺失

- 解决方案：增大cache的容量

- 优点：

- 能够有效减少容量缺失，同也会降低冲突性缺失，一举两得

- 缺点：

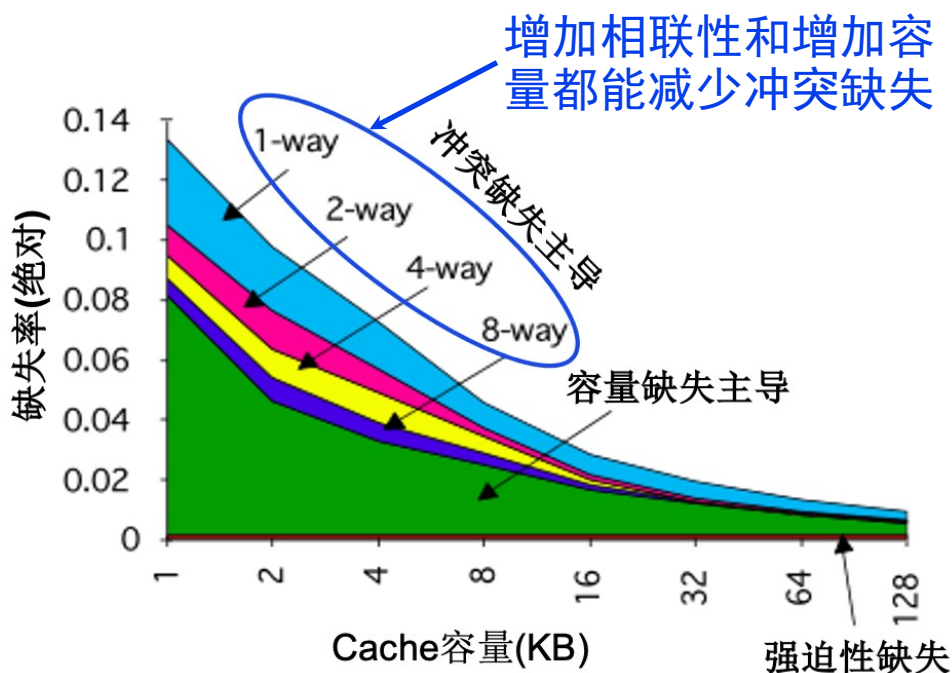
- 容量增加会造成cache的控制电路变得更加复杂，增大物理连线的延时，进而导致cache的访问时间增加。

	强制缺失
Cache容量↑	↓
Cache行大小↑	↑
Cache相联度↑	-

缓存性能评估：3C原则

- 减少冲突缺失

- 解决方案：增加相联性



	冲突缺失
Cache容量↑	↓
Cache行大小↑	↑
Cache相联度↑	↓

- 其他解决方案：增加cache的行数

Q：为什么增加cache的行数（每行大小减小，总容量不变）会有帮助？

A：可以使得相联性划分得更细，原来的cache行映射关系将会更加分散，同组内的将分散至更多的组。

缓存性能评估：3C原则

- Cache容量、行大小、相联度的影响
 - 命中时间：主要与硬件实现的复杂度有关，复杂度越高，物理连线的延时就越大，从而命中时间会更长
 - 采用控制变量法进行研究。如增大cache容量时，行大小、相联度不变，增加的是cache行数

	强制缺失	容量缺失	冲突缺失	命中时间
Cache容量↑	-	↓	↓	↑
Cache行大小↑	↓	↑	↑	↓
Cache相联度↑	-	-	↓	↑

缓存性能评估：3C原则

- 例：行大小和相联度不变，增大Cache容量
 - 强迫性缺失：不变。因为只和行大小有关
 - 容量缺失：减少。缓解了容量小的问题
 - 冲突缺失：减少。相当于增多了Cache行的数量，数据有机会被映射到更多的组中。
 - 命中时间：增大。容量增加使得电路面积增大，外围电路更多更复杂，导致延时增加

	强制缺失	容量缺失	冲突缺失	命中时间
Cache容量↑	-	↓	↓	↑
Cache行大小↑	↓	↑	↑	↓
Cache相联度↑	-	-	↓	↑

Cache性能评估

- Cache性能评估
 1. Cache缺失的3C原则
 2. Cache评价指标
 3. 如何提高Cache性能

Cache性能评估：Cache评价指标

- CPU时间可以分成两部分：
 - CPU花在**执行程序上**的时钟周期
 - CPU花在**等待存储器系统上**的时钟周期
- 通常把Cache命中时的访问开销算作CPU正常执行周期的一部分

$$\text{CPU时间} = \text{时钟周期} \times (\text{CPU执行时钟周期数} + \text{存储器停顿时钟周期数})$$

Cache性能评估：Cache评价指标

- 存储器的停顿主要是Cache缺失导致
- 存储器停顿时钟周期数可以分为两部分：
 - 读操作引起的停顿
 - 写操作引起的停顿

存储器停顿时钟周期数 =
读停顿时钟周期数 + 写停顿时钟周期数

Cache性能评估：Cache评价指标

- 读停顿时钟周期数由程序中读访问的次数、读缺失率、读缺失损失决定

$$\text{读停顿时钟周期数} = \text{读次数} \times \text{读缺失率} \times \text{读缺失损失}$$

Cache性能评估：Cache评价指标

- 写缺失导致停顿的情况
 - 对于写通过(Write Through)且缺失时先加载Cache的方式而言，写停顿主要包括：
 - 写缺失停顿：在进行写操作之前，首先要将缺失的主存储器块装入对应的Cache行中
 - 写缓冲区停顿：写操作时缓冲区已满，需要检查写缓冲区

$$\begin{aligned} & \text{写通过的停顿时钟周期数} \\ &= \text{写次数} \times \text{写缺失率} \times \text{写缺失损失} \\ & \quad + \text{写缓冲区停顿周期数} \end{aligned}$$

Cache性能评估：Cache评价指标

- 写缺失导致停顿的情况
 - 对于写回(Write Back)且缺失时先加载Cache的方式而言，写停顿主要包括：
 - 写缺失停顿：在进行写操作之前，首先要将缺失的主存储器块装入对应的Cache行中
 - 替换操作产生额外停顿

写回的停顿时钟周期数
= 写次数 × 写缺失率 × 写缺失损失
+ 替换操作停顿周期数

Cache性能评估：Cache评价指标

- 对于缺失时先加载Cache行的Cache而言，读和写的缺失损失是相等的，都是将缺失的块装入Cache所花的时间
- 对于精心设计的Cache系统而言，写缓冲区停顿、替换停顿均可忽略

$$\text{存储器停顿时钟周期数} = \text{访问次数} \times \text{缺失率} \times \text{缺失损失}$$

Cache性能评估：Cache评价指标

- **例1：**已知处理器基本CPI = 2，在如下条件下计算理想情况下CPU性能是实际情况下的多少倍
 - 执行程序时指令Cache缺失率为2%，数据Cache缺失率为4%
 - 缺失损失为100个时钟周期，访问数据存储器指令的比例为36%
- **解答：**设程序的指令数为I
 - 指令缺失时钟周期数： $I \times 2\% \times 100 = 2I$
 - 数据缺失时钟周期数： $I \times 36\% \times 4\% \times 100 = 1.44I$
 - 因此，实际CPI = $2 + 2 + 1.44 = 5.44$
 - 理想机器的性能与实际机器的倍数关系：

$$\frac{\text{实际CPU时间}}{\text{理想CPU时间}} = \frac{I \times \text{CPI}_{\text{实际}} \times \text{时钟周期}}{I \times \text{CPI}_{\text{理想}} \times \text{时钟周期}} = \frac{\text{CPI}_{\text{实际}}}{\text{CPI}_{\text{理想}}} = \frac{5.44}{2} = 2.72$$

Cache性能评估：Cache评价指标

- **例2：**例1的基本CPI由2减为1，其他条件不变。计算理想情况下CPU性能是实际情况下的多少倍
- **解答：**设程序的指令数为I
 - 指令缺失时钟周期数： $I \times 2\% \times 100 = 2I$
 - 数据缺失时钟周期数： $I \times 36\% \times 4\% \times 100 = 1.44I$
 - 因此，例2的实际CPI = $1 + 2 + 1.44 = 4.44$
 - 理想机器的性能是例2机器的： $4.44 / 1 = 4.44$ 倍
 - 例2机器的性能是例1机器的： $5.44 / 4.44 = 1.23$ 倍

CPI减半，但是性能没有提升一倍！

Cache性能评估：Cache评价指标

- **例3：**例1的处理器时钟频率加倍，其他条件不变（访问存储器的绝对延时不变）。计算例3中的CPU性能是例1的多少倍
- **解答：**设程序的指令数为I
 - 指令缺失时钟周期数： $I \times 2\% \times 200 = 4I$
 - 数据缺失时钟周期数： $I \times 36\% \times 4\% \times 200 = 2.88I$
 - 因此，例3的实际CPI = $2 + 4 + 2.88 = 8.88$
 - 例3机器的性能与例1机器的倍数关系：

$$\frac{\text{例1机器CPU时间}}{\text{例3机器CPU时间}} = \frac{I \times \text{CPI}_{\text{实际1}} \times \text{时钟周期}}{I \times \text{CPI}_{\text{实际3}} \times \text{时钟周期}/2} = \frac{\text{CPI}_{\text{实际1}}}{\text{CPI}_{\text{实际3}}/2} = \frac{5.44}{8.88/2} = 1.23$$

频率翻倍，但是性能没有提升一倍！

Cache性能评估

- Cache性能评估
 1. Cache缺失的3C原则
 2. Cache评价指标
 3. 如何提高Cache性能

Cache性能评估：如何提高Cache性能

- Cache缺失率和相联度之间的关系
 - 2路组相联相比于1路组相联(直接映像)的数据缺失率下降的比例约为15%
 - 更高的相联度对数据缺失率的改进有限

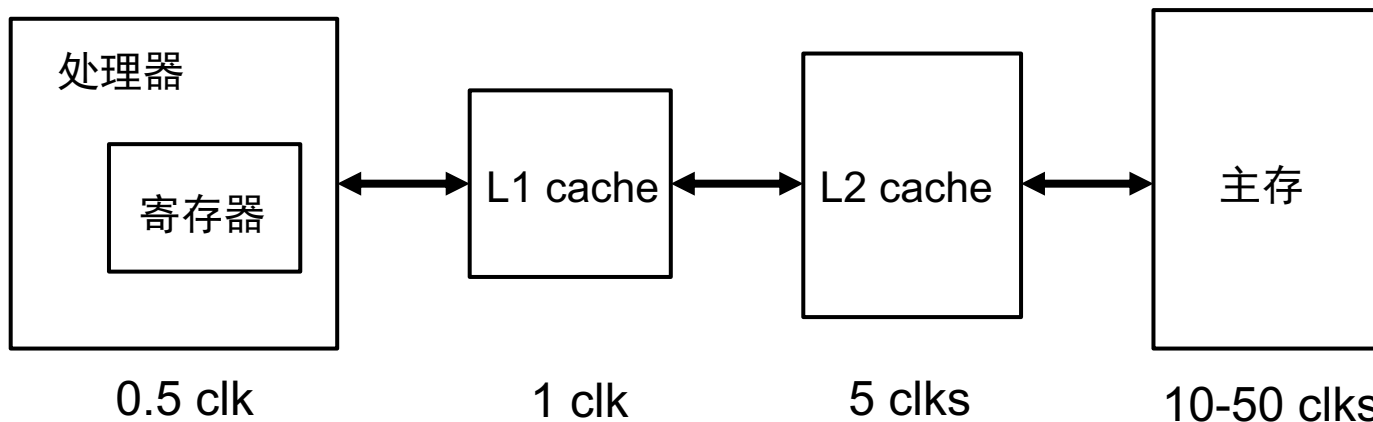
相联度	数据缺失率
1	10.3%
2	8.6%
4	8.3%
8	8.1%

SPEC2000基准测试的数据缺失率

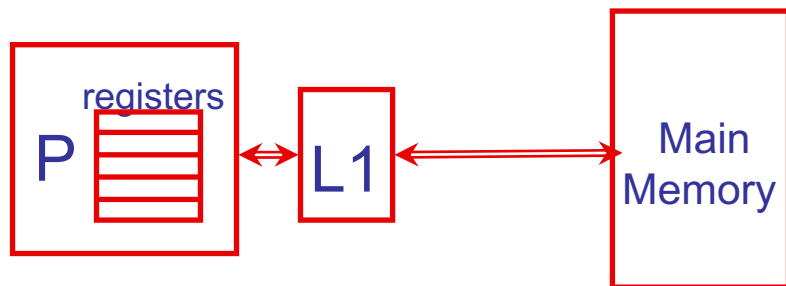
Cache性能评估：如何提高Cache性能

- 多级cache策略

- 为进一步缩小现代处理器高时钟频率与DRAM相对较慢之间的差距，处理器多采用多级Cache
- 以两级Cache为例
 - 如果访问L1 Cache时缺失，则访问L2 Cache
 - 如果L2 Cache包含所请求的数据，则缺失损失就是L2 Cache的访问时间，这要比主存的访问时间少得多



缓存性能的分析：二级缓存



访问时间

1 clk

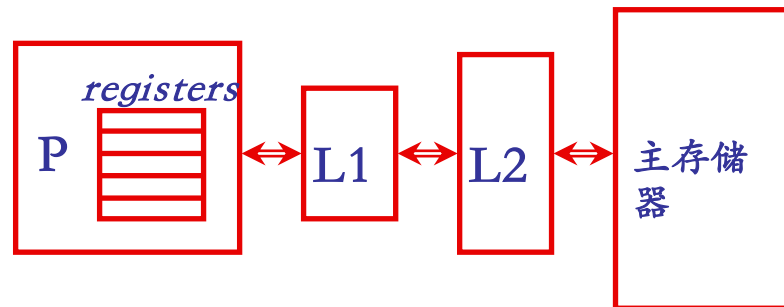
10 clks

$$\text{平均访问时间} = t_h + (1-h)t_m$$

假定 $h_{L1} = 70\%$

$$\text{平均访问时间} = 1 + 0.30 \times 10 = 4 \text{ clks}$$

(比没有cache时强)



访问时间

1 clk

5 clks

10 clks

$$\text{平均访问时间} = t_{L1} + (1-h_{L1})(t_{L2} + (1-h_{L2})t_m)$$

$h_{L2} = 90\%$ $L1$ 平均访问时间 $L2$ 平均访问时间

$$\begin{aligned} \text{平均访问时间} &= 1 + 0.30 \times (5 + 0.10 \times 10) \\ &= 1 + 0.3 \times 6 = 2.8 \text{ clks} \\ &\text{(better than L1 only - 4 clks)} \end{aligned}$$

Cache性能评估：如何提高Cache性能

- 多级cache策略
 - 以两级Cache为例，L1和L2 Cache的设计思想不同：
 - L1 Cache致力于减少命中时间，以获得较短的时钟周期
 - L2 Cache致力于降低缺失率，以减少访问主存的缺失损失
 - 由于L2 Cache的存在，L1 Cache缺失造成的损失大大减少，缺失率高一些也没关系。所以L1 Cache通常较小，以减少命中时间。
 - 由于L1 Cache的存在，L2 Cache的命中时间仅影响L1 Cache的缺失损失，不直接影响L1 Cache的命中时间和处理器时钟周期。所以L2 Cache通常较大，块容量也比L1 Cache大

Cache性能评估：Cache评价指标

- **例：**对于如下参数的存储系统，请计算用两级cache的速度是一级cache速度的多少倍。
 - 已知处理器的基本CPI=1，时钟频率为5GHz
 - 访问一次主存的时间为100ns
 - 假设L1 cache缺失率为2%，对L1 cache访问仅需一个周期
 - L2 cache的命中和缺失访问时间都是5ns，其缺失率仅为0.5%
- **解答：**
 - 处理器时钟频率为5GHz，则时钟周期为0.2ns
 - 访问一次主存的周期损失为： $100\text{ns} / 0.2\text{ns} = 500$
 - 访问L2 cache的周期损失为： $5\text{ns} / 0.2\text{ns} = 25$
 - 情况一：只有一级cache
$$\text{CPI}_1 = 1 + 2\% \times 500 = 11$$
 - 情况二：有两级cache
$$\text{CPI}_2 = 1 + 2\% \times (25 + 0.5\% \times 500) = 1.55$$
 - 因此采用两级cache相比于一级cache的处理器速度倍数为： $11 / 1.55 = 7.10\text{倍}$

Cache性能评估：如何提高Cache性能

- 关于Cache性能的三点总结：
 - Cache的性能
 - 处理器速度加快(降低CPI or 提高时钟频率), 存储器阻塞周期的影响也相对增加
 - Cache的容量、行大小会影响其性能
 - 利用组相联降低缺失率
 - 降低缺失率 vs. 比较器成本
 - 利用多级Cache策略降低缺失损失
 - 成本和工艺的限制以及时钟频率的极限使得一级Cache不能做的更大

小结

- 存储系统基础：半导体存储器
 - SRAM、DRAM的基本电路、读写流程及各自特点
 - 存储器的三个基本参数
- 存储系统的设计理论
 - 局部性原理：时间局部性、空间局部性
 - 存储器层次结构及其性能指标

小结

- **Cache技术**

- Cache基本结构、Cache存储体结构
- Cache的地址映像机构，三种地址映像方式：直接相联映像、全相联映像、组相联映像
- 读数据时的替换操作：随机法、FIFO、LRU
- 写数据时的更新操作：写通过、写回

- **Cache性能评估**

- 3C原则：强迫性缺失、容量缺失、冲突缺失
- Cache的评价指标
- 多级cache策略简介



数字逻辑与处理器基础

Fundamentals of Digital Logic and Processor



第八讲 存储器与总线外设

第2-2部分 总线和外设

李学清

#腾讯会议： 453-1488-6548

助教：唐文骏*/陈仲豪/曾令安

*本节内容负责答疑助教

2024年12月23日

本节课内容和目标

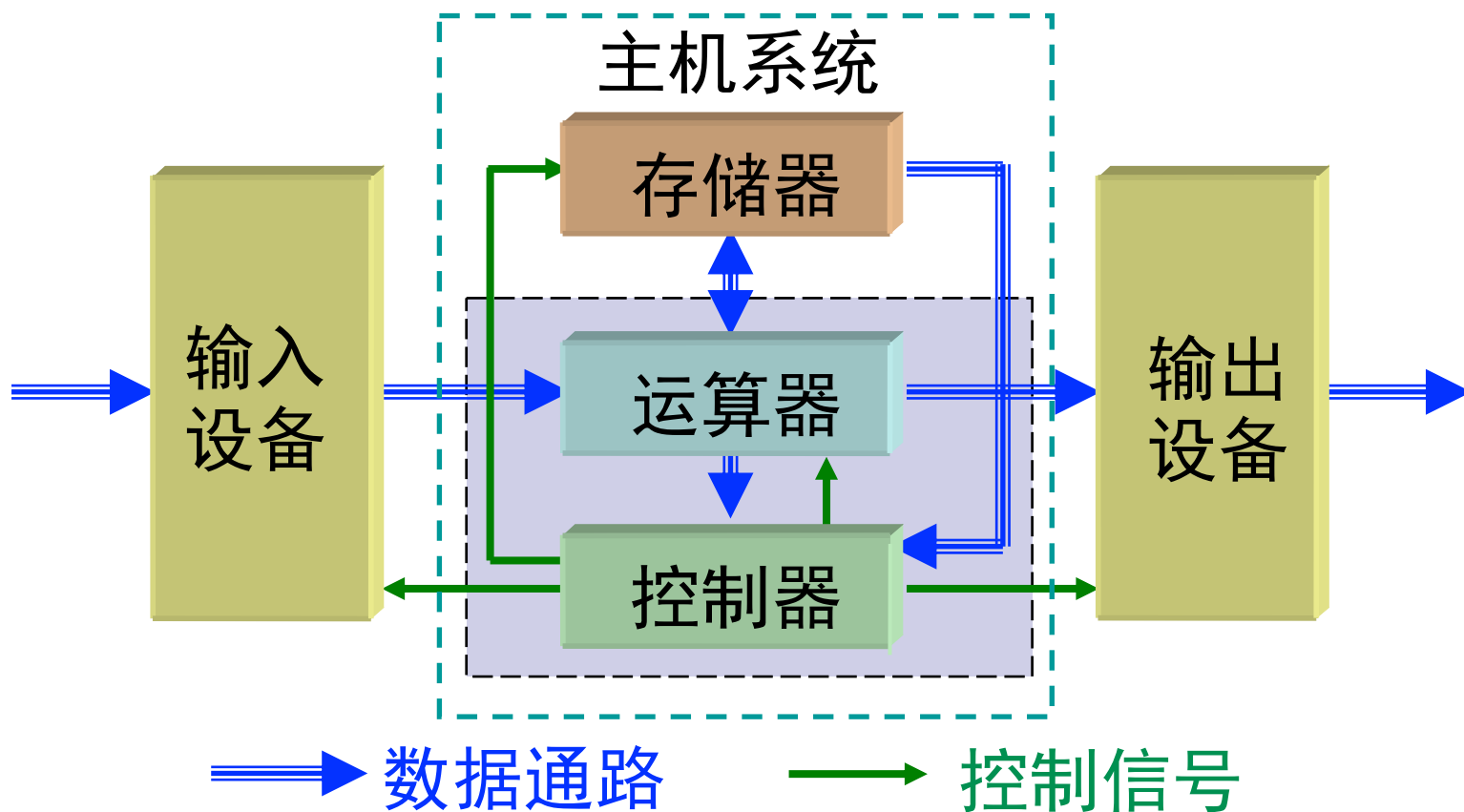
- 知识点：概念、原理和指标
 - 基本总线拓扑结构、分类、数据I/O和外围连接的应用
- 能力
 - 用总线连接多个设备的基本原则和方法
- 思想
 - 时空：信息（数据、控制等）互连和权衡
- 不重点考察，可能会考察相关简单概念，但不会考察相关计算和具体实现细节

目录

- 引言
- 总线的拓扑结构及分类
- 总线通信原理
- 外设工作原理

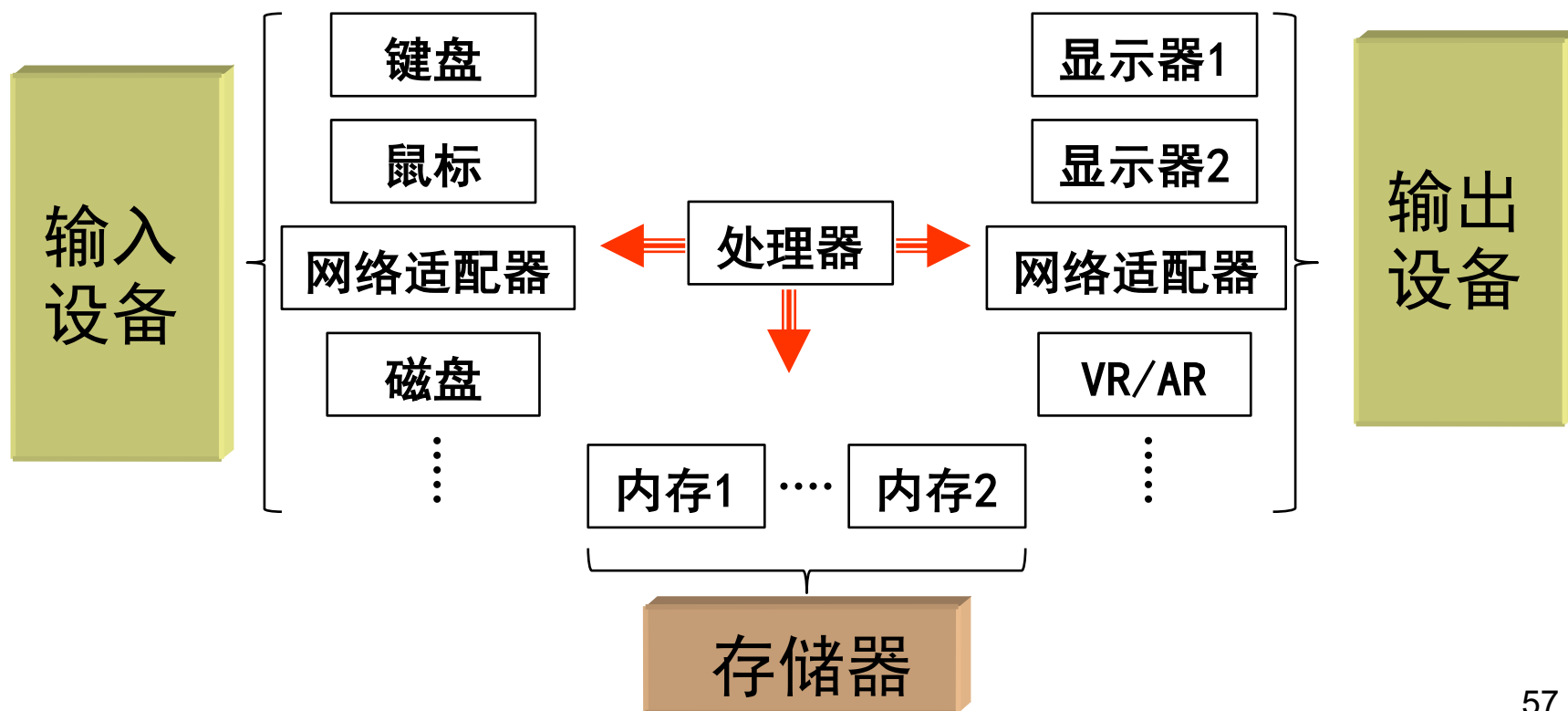
知识回顾

- 典型的计算机体系架构
 - 冯·诺伊曼架构



引言

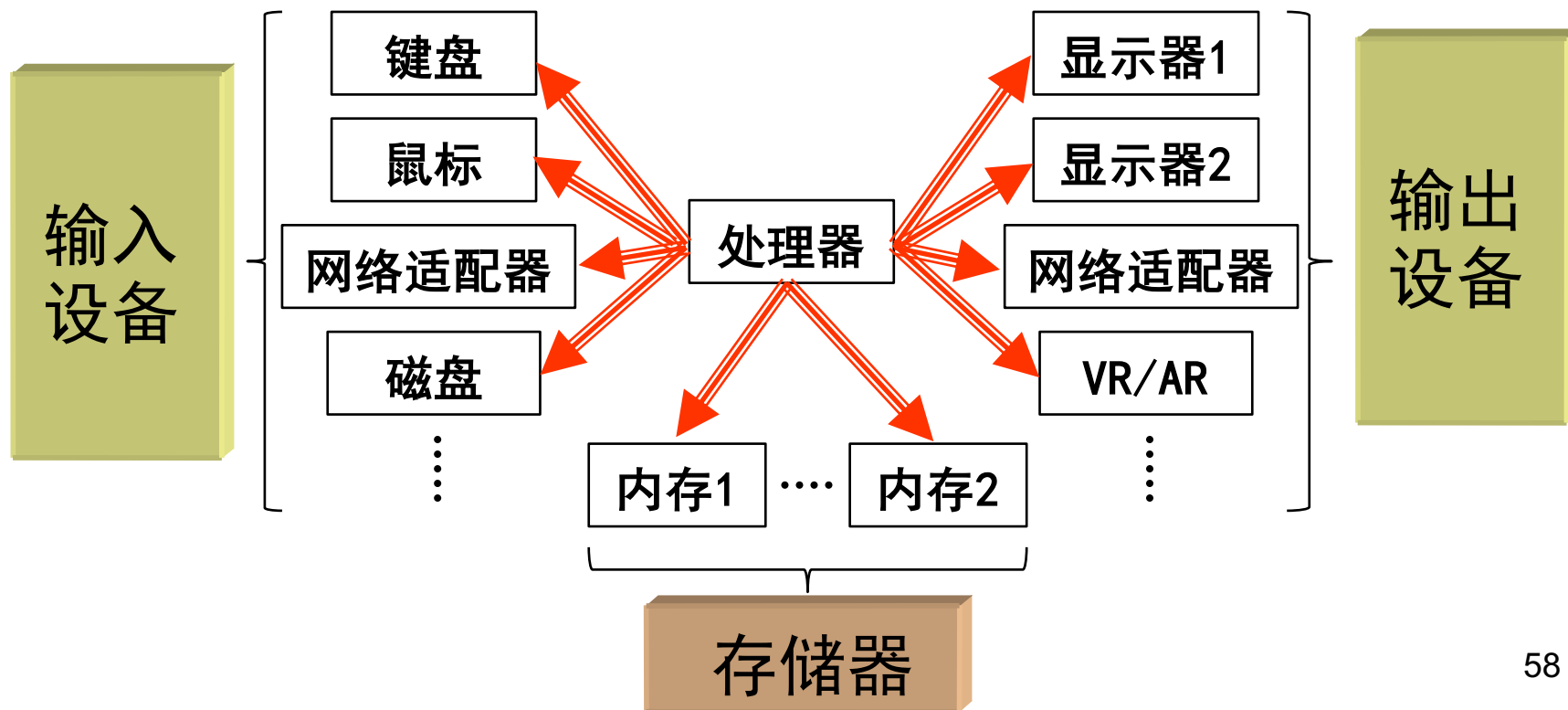
- 处理器的外部设备种类，数量繁多
 - 处理器如何和这些组件进行高效通信？



引言

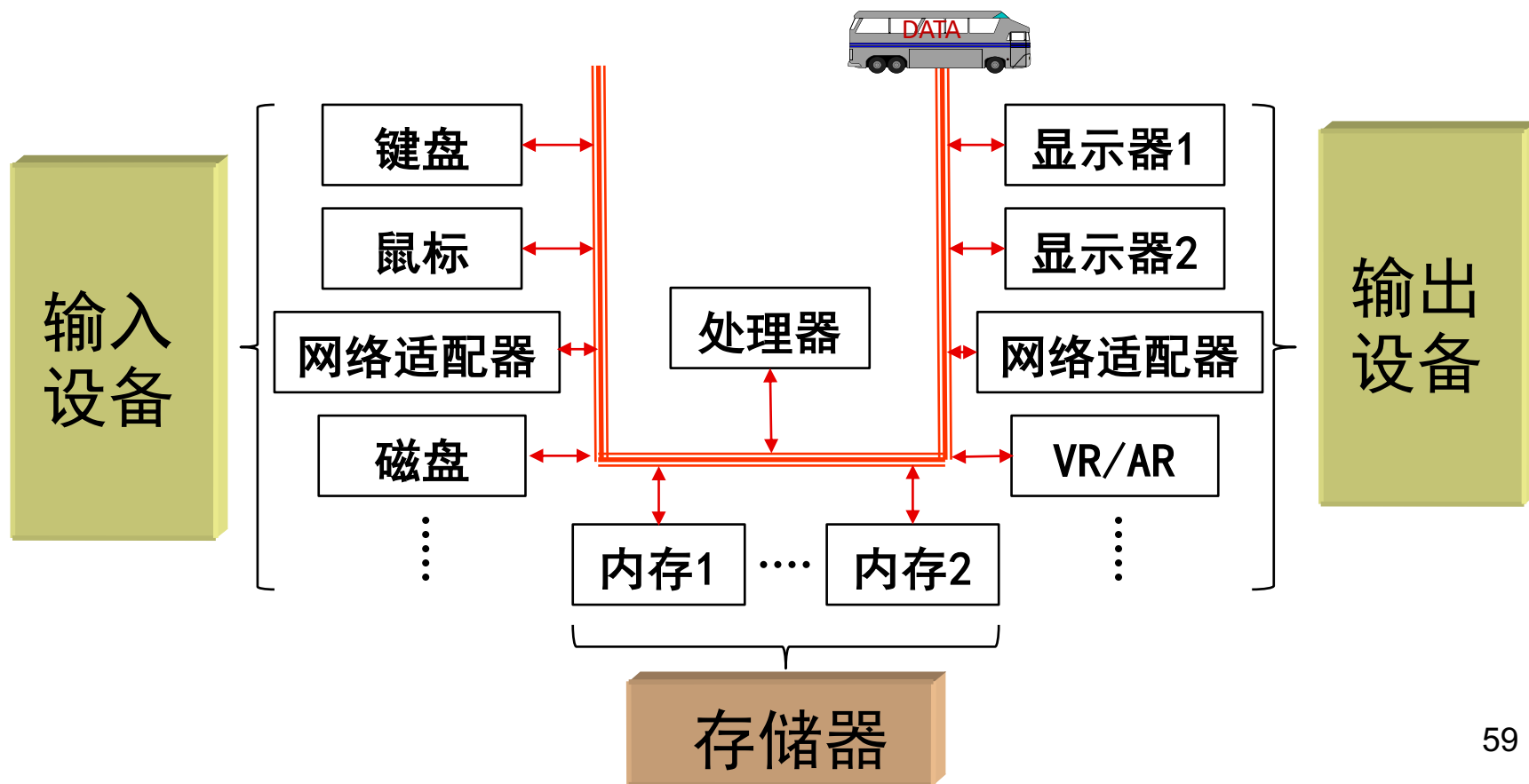
- 分散连接

- 处理器和每个设备都有独立的通信通道
- 布局布线困难，可扩展性差



引言

- 所有外部设备接在统一的接口（总线）上
 - 布局布线简单，可扩展性强



目录

- 引言
- 总线的基本结构和分类
- 总线的工作原理
- 外设的工作原理

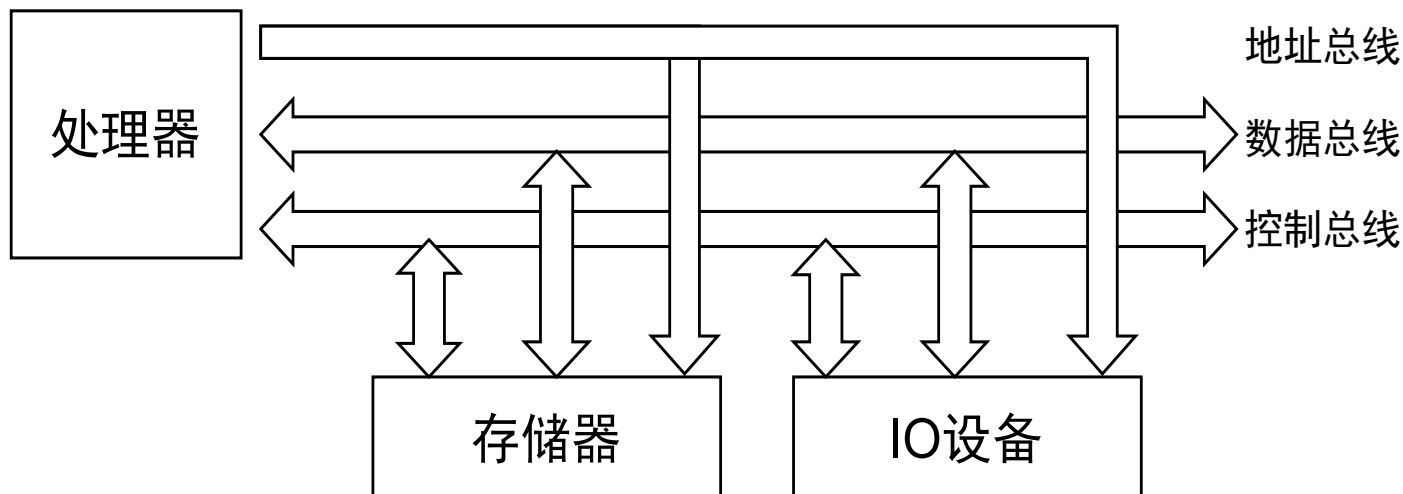
总线的基本结构和分类

- 总线是什么？

- 总线（Bus）指在计算机系统内，各个部件传递信息（如交换数据、传送控制信号）使用的一组或多组公共连接线（通信干线）

- 总线做什么？

- 总线负责在设备间传输**数据**，以及传输和数据相关的**地址信号**和**控制信号**



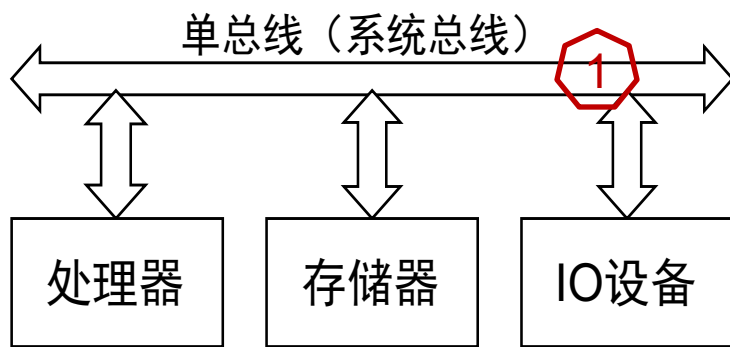
总线的基本结构和分类

- 总线的基本参数：
 - **总线宽度**：也称为总线位宽，指总线中传输线的数量
 - **总线传输频率**：总线上数据传输时的频率
 - **标准传输率**：也称为总线带宽，指总线上单位时间能够传输的最大数据量。
 - **负载能力**：在保证总线正常工作的前提下，总线能够挂载的设备数目

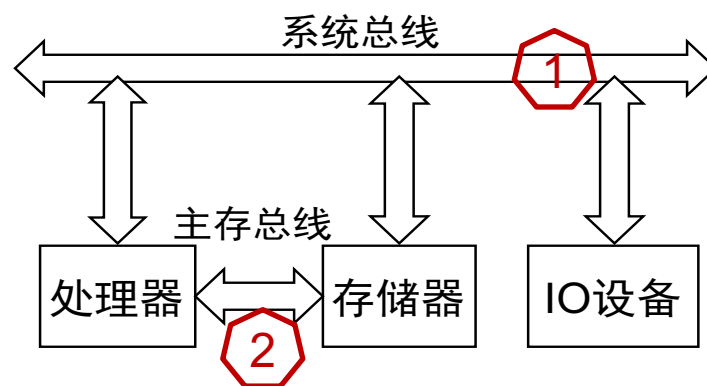
$$\text{总线带宽} = \text{总线数据传输频率} \times \text{总线位宽}$$

总线的基本结构和分类

- 总线可以按照总线数量划分为：
 - **单总线结构**：有且仅有一条总线
 - **双总线结构**：有且仅有两条总线
 - **多总线结构**：存在三条或三条以上总线



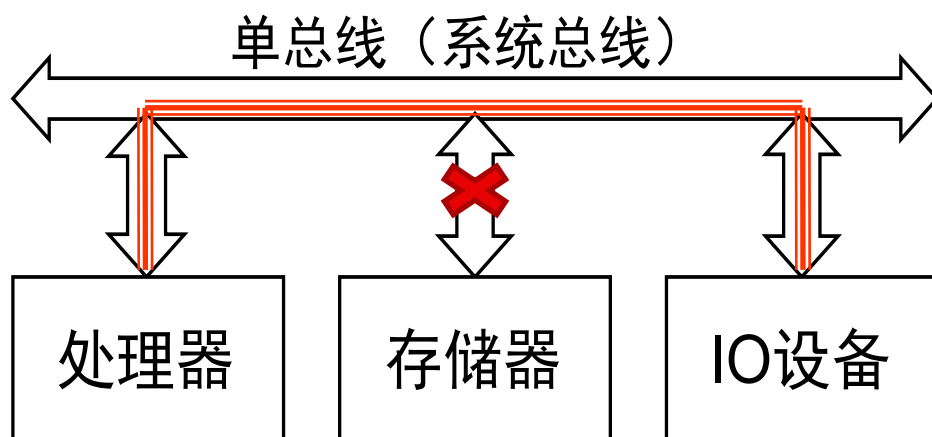
单总线结构



双总线结构

总线的基本结构和分类

- 单总线结构：
 - 所有计算机组件均连接在同一条总线上
 - 优点：结构简单，易于扩展
 - 缺点：不同速度设备间相互影响，整体通信效率低

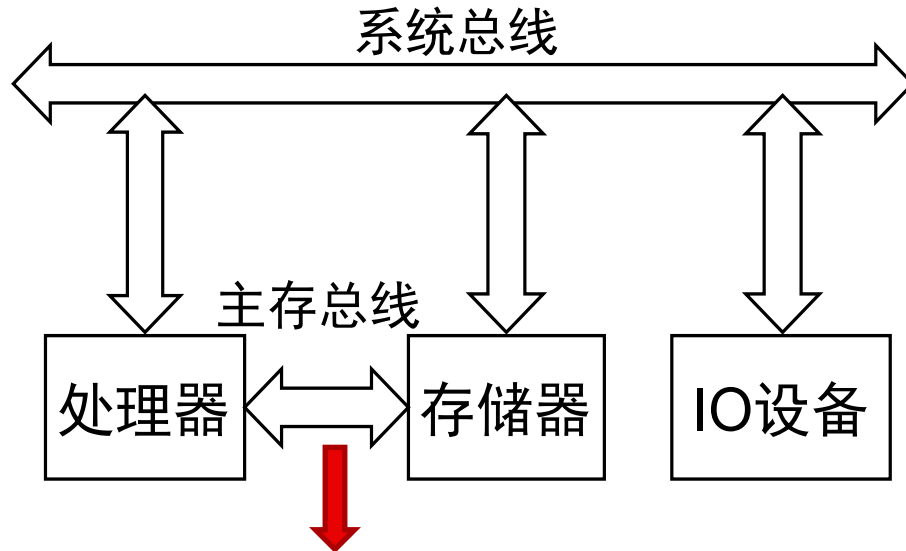


IO设备操作耗时，
会长时间占据总线，
其他设备无法使用总线
整体通信效率低

总线的基本结构和分类

- 双总线结构：

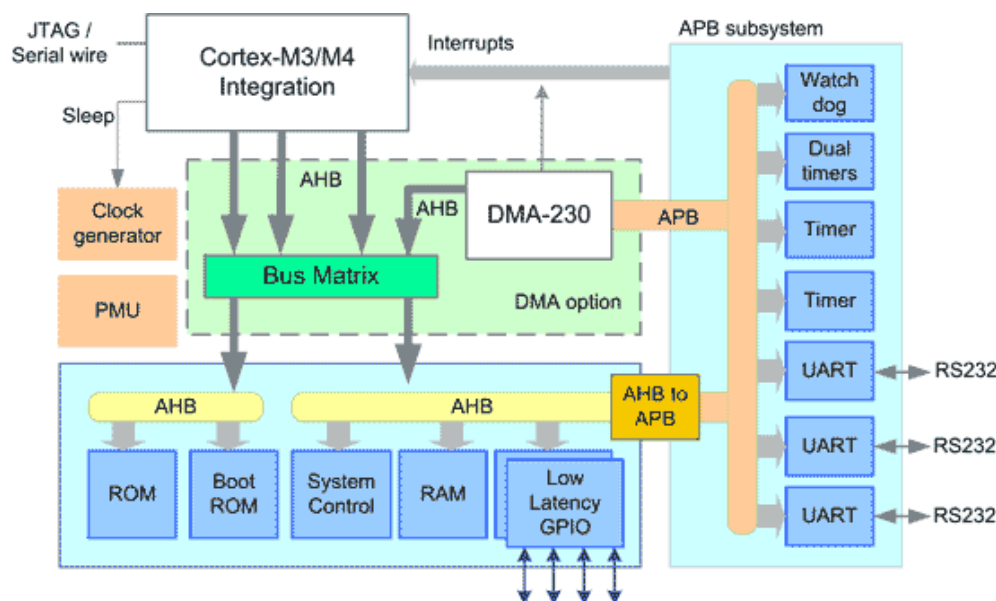
- 除了系统总线外，分离出速度较快的存储器和处理器之间的总线，作为独立的**主存总线**
- 保证了存储器和处理器间的高效通信



和系统总线独立，
且并行的主存总线

总线的基本结构和分类

- 多总线结构：
 - 单总线和双总线往往不能满足当今计算机系统的高效通信需求，一般采用多总线结构



ARM Cortex-M3处理器总线示意图

AHB: 高级高性能总线

APB: 高级外围总线

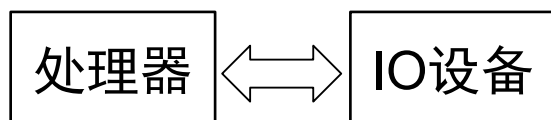
除此之外还有常用的AXI（高级可拓展接口）总线

目录

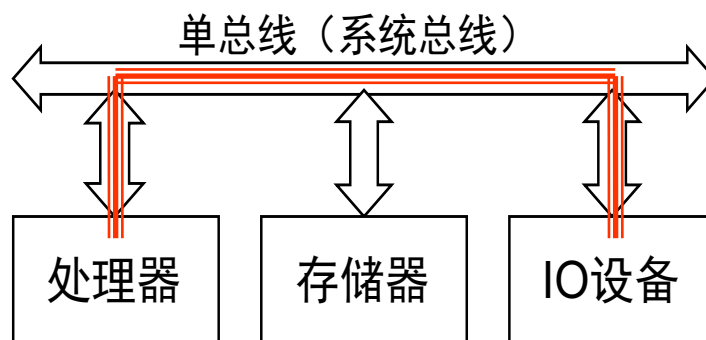
- 引言
- 总线的基本结构和分类
- 总线的工作原理
- 外设的工作原理

总线的工作原理

- 基于总线进行数据传输面临的挑战：
 - 总线上任意两个设备上都可能发生数据传输
 - 一般地，主动发起基于总线的数据传输成为**主设备**，另外一个设备称为**从设备**
 - **主线上的每个设备都有自己独一无二的设备地址**
 - 总线可能正在被其他设备占用
 - 总线一般采用**申请-使用**流程，总线空闲时才能使用



点对点通信
有需要即可使用



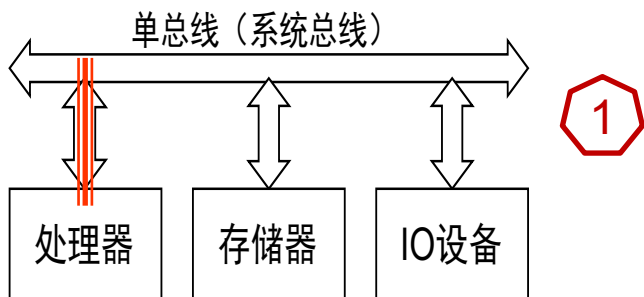
基于总线的通信
先申请，后使用

总线的工作原理

- 总线信息传输的四个阶段
 - **申请阶段**：主设备请求占用总线，由**总线仲裁器**决定哪个主设备可以占用总线
 - **寻址阶段**：主设备占用总线，发送从设备地址，启动从设备
 - **传输阶段**：主设备占用总线，和从设备进行数据传输
 - **结束阶段**：主设备停止占用总线
 - 以上四个阶段构成一个完整的**总线传输周期**

总线的工作原理

• 总线信息传输的四个阶段示例



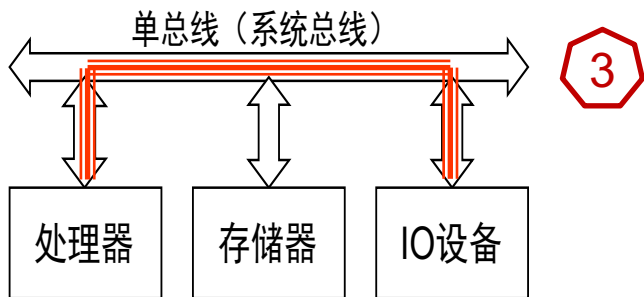
处理器向总线仲裁器申请:

处理器申请使用主线

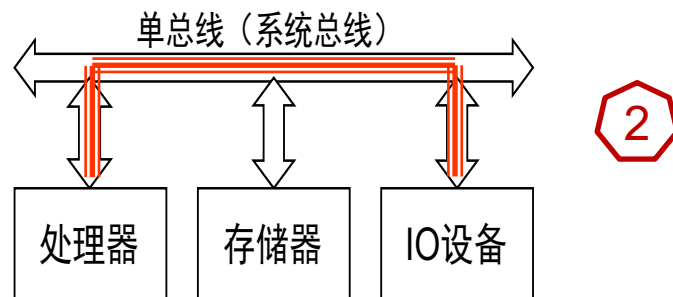
总线仲裁器发现总线空闲:

处理器可以使用总线

申请阶段



传输阶段



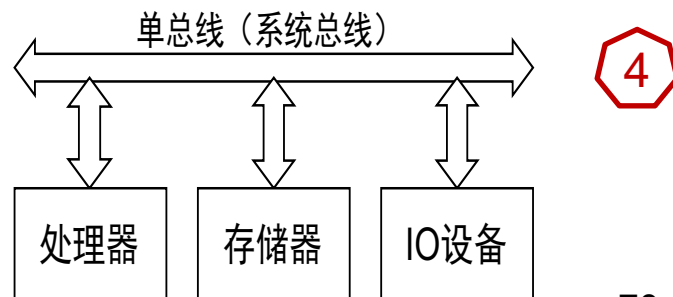
处理器准备启动从设备 (IO设备):

启动IO设备 (0x01)

IO设备监听总线并回复:

IO设备 (0x01) 已经启动

寻址阶段



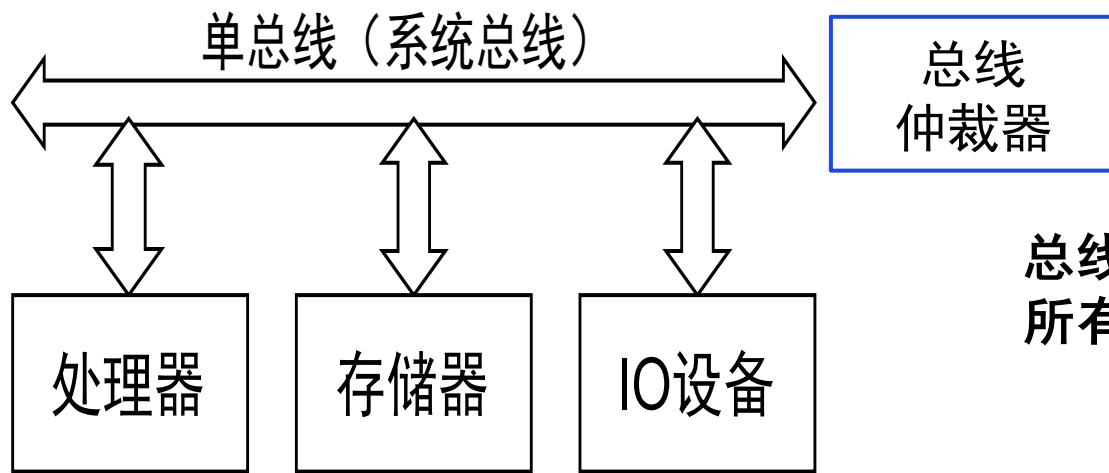
结束阶段: 停止占用

总线的工作原理

- 总线仲裁器是什么？
 - 总线上可能同时有多个设备申请占用总线，总线仲裁器决定下一个总线传输周期哪个设备占用总线
 - 设备1和设备2同时申请占用总线，仲裁器决定设备1占用，于是设备1开始占用，设备2继续等待
 - 根据仲裁器所在的位置可以分为：集中式仲裁系统和分布式仲裁系统

总线的工作原理

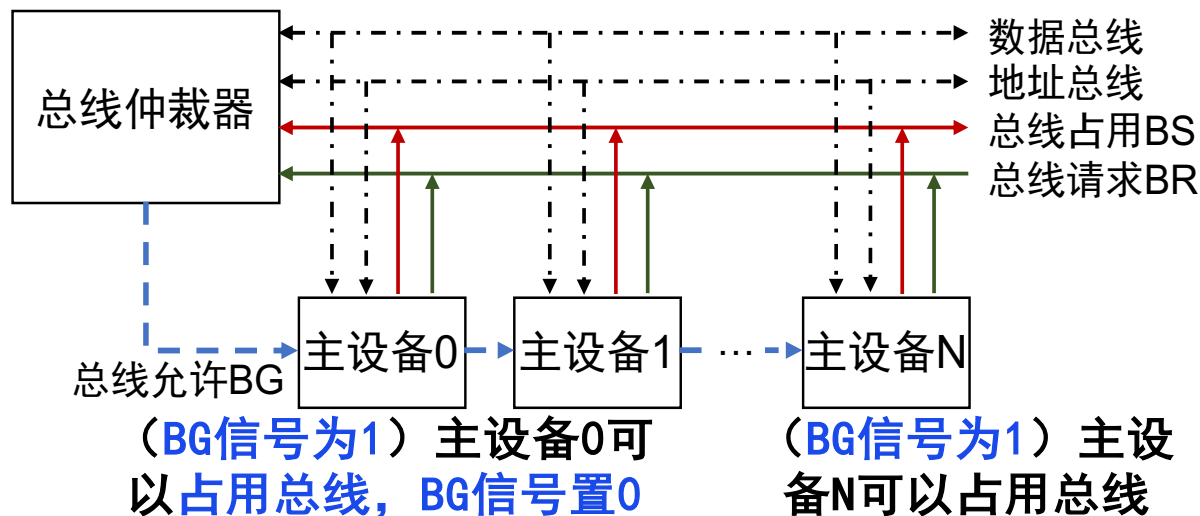
- 集中式仲裁系统：
 - 总线判优的控制逻辑**集中**在全局位置上的总线仲裁器
 - 可以划分为三大类：**链式查询**，**计数器查询**以及**总线独立请求与准许**



总线仲裁器统一处理
所有设备的总线请求

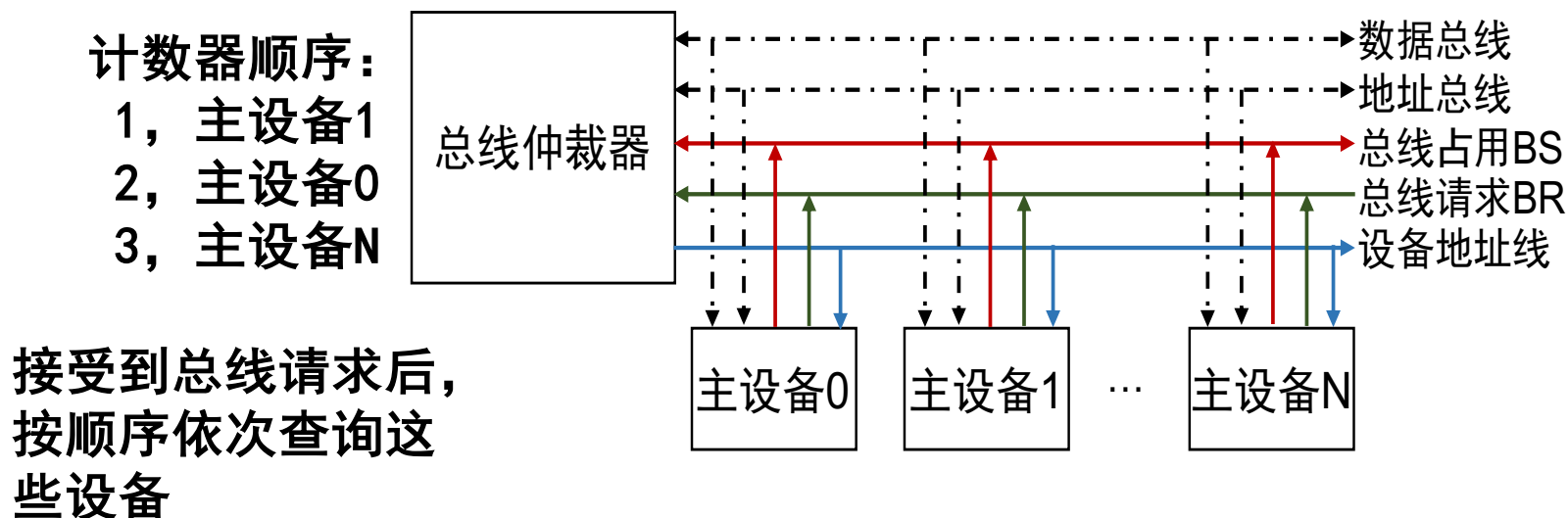
总线的工作原理

- 集中式仲裁系统（链式查询）：
 - 按照**硬件顺序**依次查询是否有总线申请
 - 设备使用总线的优先级由硬件结构确定
 - 例子中，主设备0**优先级最高**，主设备N**优先级最低**
 - 信号线说明：**总线请求** (BR, bus request)、**总线占用** (BS, bus busy) 以及**总线允许** (BG, bus grant)



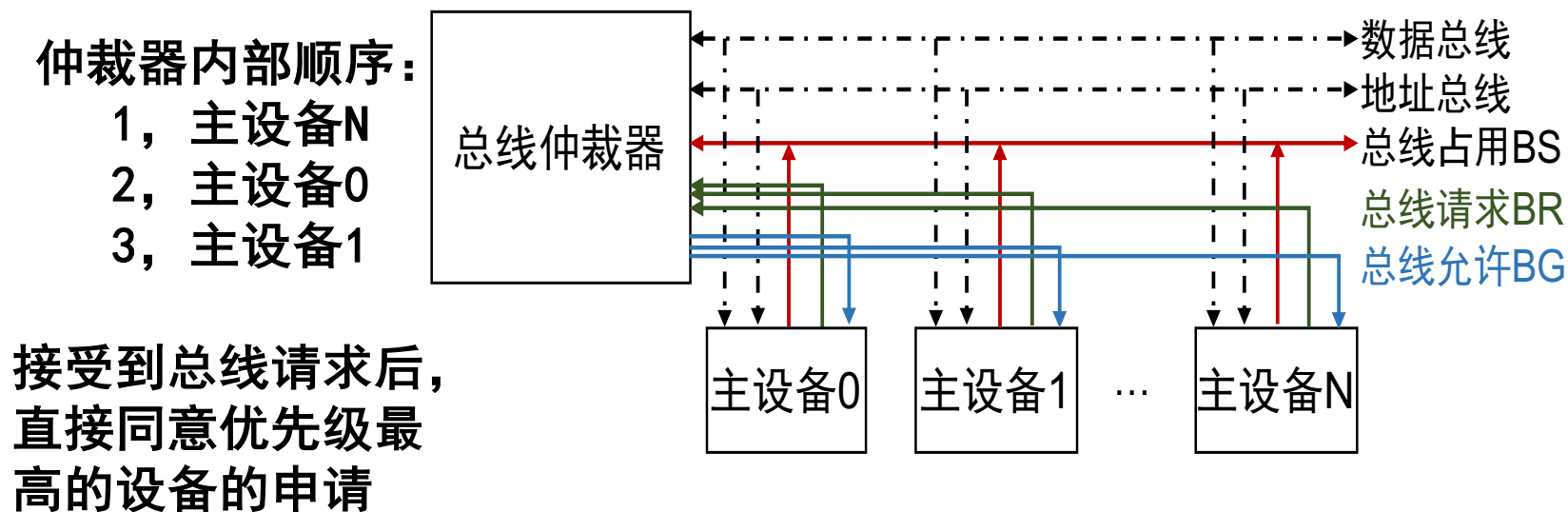
总线的工作原理

- 集中式仲裁系统（计数器查询）：
 - 在有总线申请后，按照**计数器**顺序逐步查询设备
 - 设备使用总线的优先级由计数器顺序确定
 - 例子中，主设备1**优先级最高**，主设备N**优先级最低**



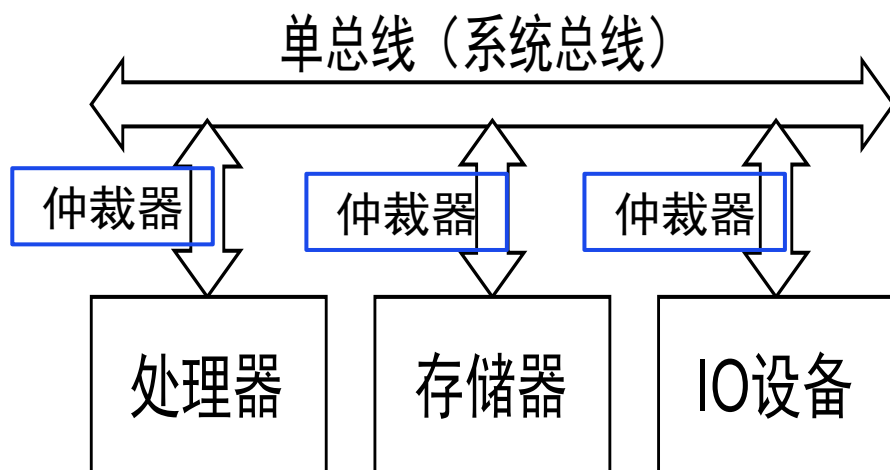
总线的工作原理

- 集中式仲裁系统（总线独立请求与准许）：
 - 每个设备和总线仲裁器均有独立的连接
 - 设备使用总线的优先级由仲裁器内部设定决定
 - 例子中，主设备N**优先级最高**，主设备1**优先级最低**



总线的工作原理

- 分布式仲裁系统：
 - 总线判优的控制逻辑**分散**在与总线相连的各个设备上
 - 每个设备都有独立的仲裁器判断本身设备能否占用总线



当处理器和存储器同时申请占用总线
两个仲裁器比较自己的申请优先级
优先级高的占用，优先级低的退出

目录

- 引言
- 总线的基本结构和分类
- 总线的工作原理
- 外设的工作原理

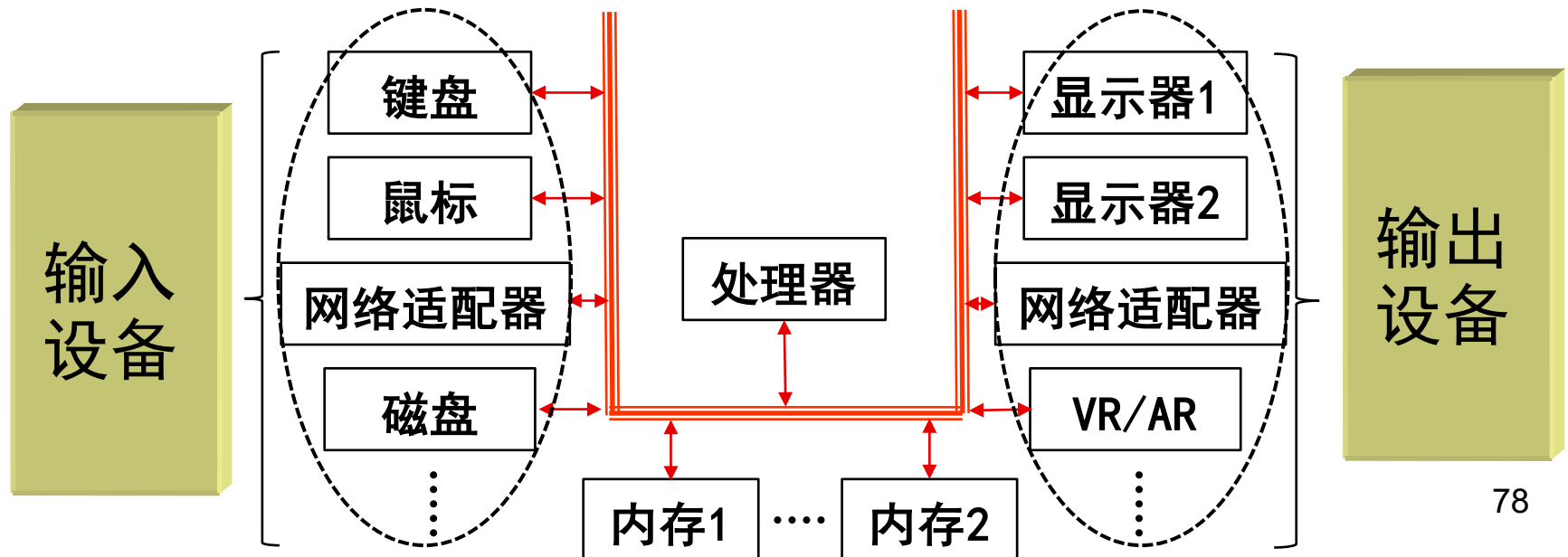
外设的工作原理

- 外设是什么？

- 外设是外部设备的简称，是指连接在计算机系统中，处理器以外的硬件设备

- 外设做什么？

- 外设用来扩展或增强所连接的计算机系统的功能或性能



外设的工作原理

- 常见的外设
 - 键盘，鼠标，显示器，固态硬盘等



外设的工作原理

- 外设可以按照功能划分为：

- 输入设备：键盘，鼠标等
- 输出设备：显示器等
- 存储器设备
- 网络设备



外设的工作原理

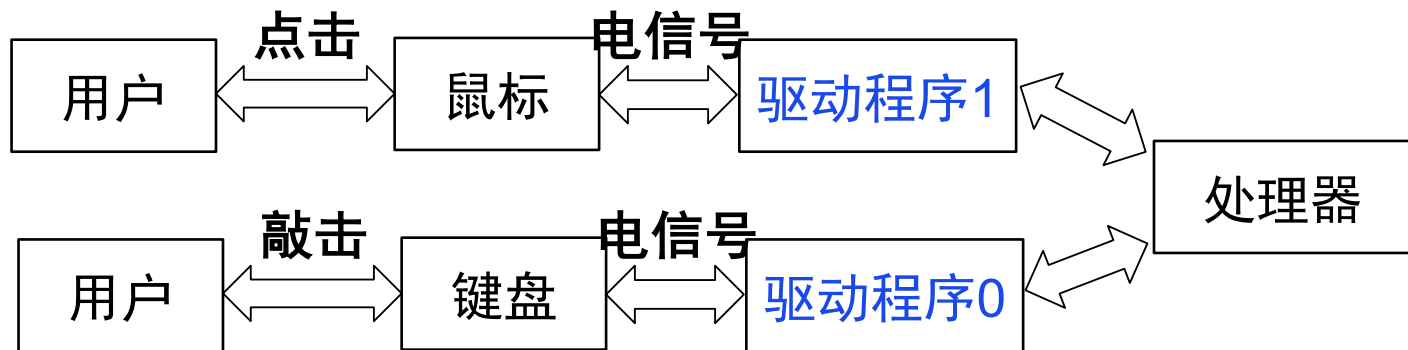
- 通过外设扩展处理器功能面临的挑战：
 - 外设和外部环境的交互信息，外设和处理器的交互信息，两者往往是不同的
 - 例如：对于键盘，我们和键盘交互采用的是物理信号（敲击按键），键盘和处理器交互是数字信号
 - 因此需要程序来进行两种不同信息的转换， 这类应用程序也被称为**驱动程序**



此图为示意图
实际结构更复杂

外设的工作原理

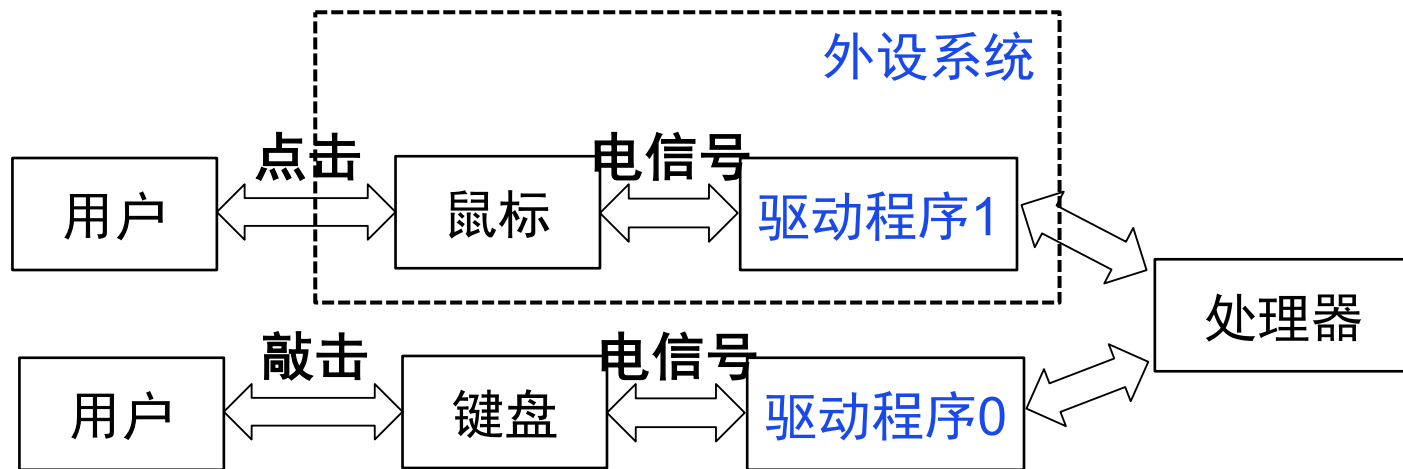
- 通过外设扩展处理器功能面临的挑战：
 - 不同外设，交互信息多样
 - 例如：104键键盘和87键键盘同样按‘Q’键产生的信息不同
 - 因此，每一种外设都要有**对应的驱动程序**



此图为示意图
实际结构更复杂

外设的工作原理

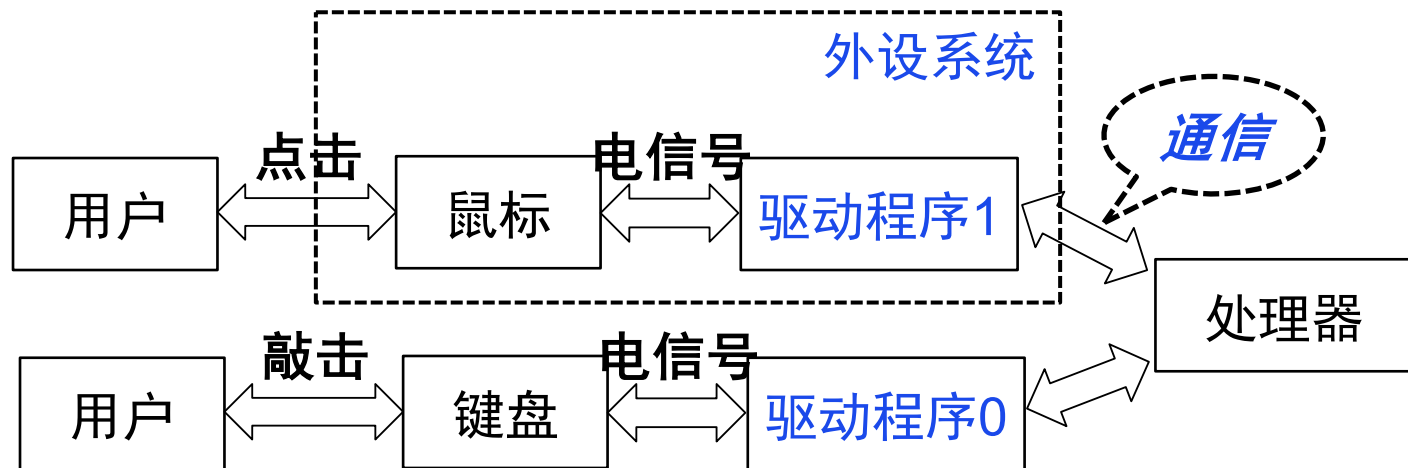
- 外设和对应的驱动程序共同组成外设系统
 - 对数据和信息起着传输、转送和存储的作用
 - 是计算机系统中的重要组成部分



此图为示意图
实际结构更复杂

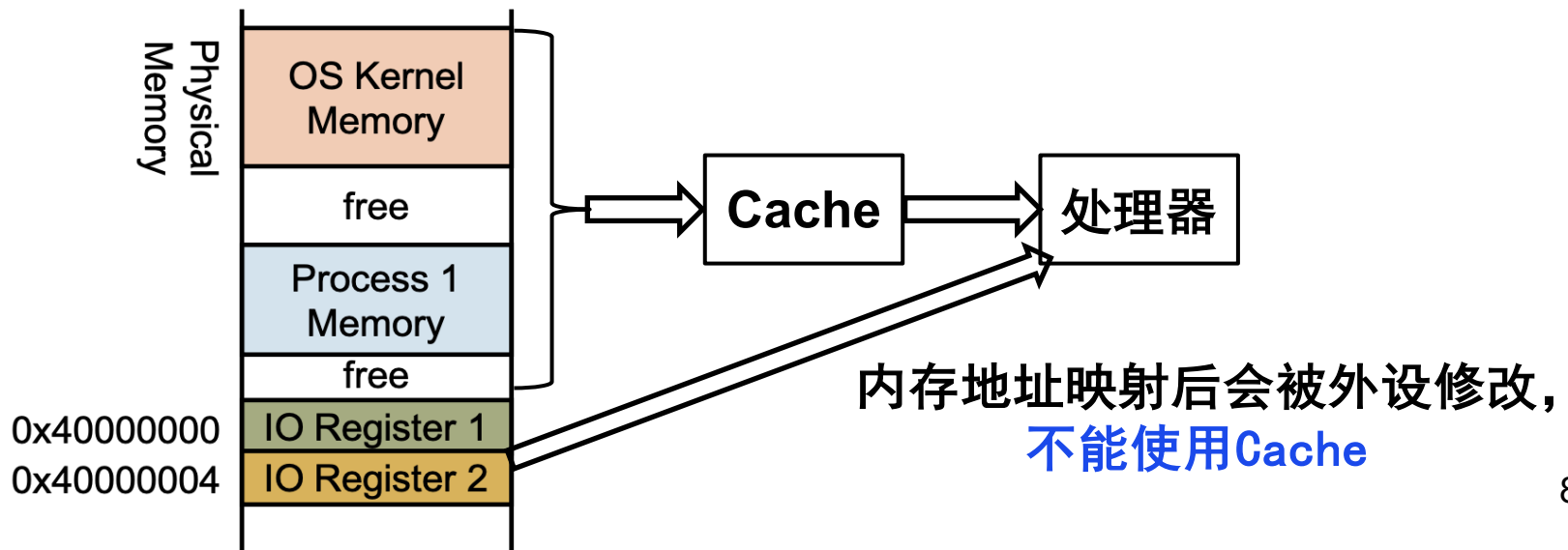
外设的工作原理

- 外设系统的一个重要功能就是和处理器进行通信
 - 途径：通过什么和处理器进行通信？
 - 方式：和处理器采用怎样的方式通信？



外设的工作原理

- 通过什么和处理器进行通信：
 - **特殊指令访问**，例如x86指令集中的in和out指令
 - **内存映射地址访问**，将某段内存映射为某一外设和处理器通信的专用内存（主流方案）
 - 例如0x40000000为IO设备1的专用内存，IO设备通过改写内存数据来告知处理器自身产生的信息



外设的工作原理

- 和CPU采用怎样的方式通信：
 - 轮询式，设置状态信号，处理器周期性查看状态信号



ready为状态信号，为1时代表数据准备完成

- 中断式**，设置中断信号，当中断信号到来时，中断处理器当前程序，执行中断程序



full为中断信号，为1时触发处理器中断

总线和外设总结

- 总线

- **是什么**：总线是计算机系统内，各个部件传递消息使用的一组或多组公共连接线
- **有哪些**：总线可以按照数量划分为单总线结构，双总线结构以及多总线结构
- **工作流程**：总线传输的四个阶段，以及总线仲裁器的分类及原理

总线和外设总结

- 外设

- **是什么**：外设是指连接在计算机系统中，处理器以外的硬件设备
- **有哪些**：输入设备、输出设备、存储器设备以及网络设备
- **工作流程**：外设和对应驱动程序组成外设系统。外设系统和CPU交互通道一般采用内存映射地址访问，采用轮询式或中断式处理外设信息



附录

自学（不要求）

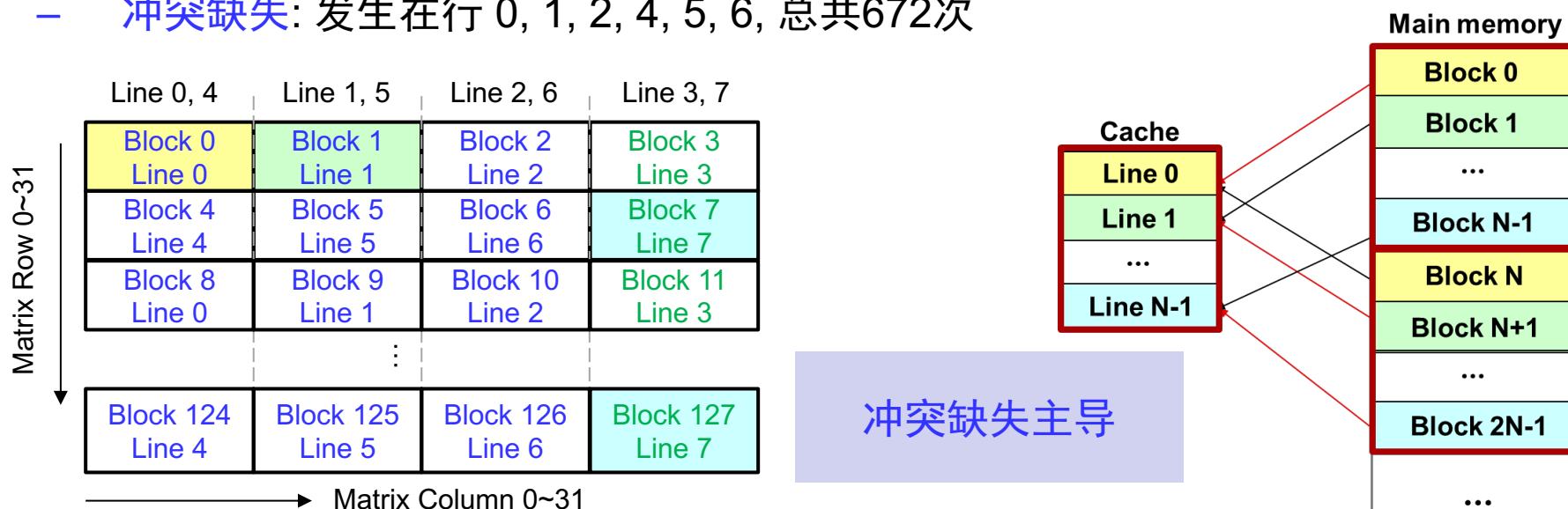
缓存缺失: 3C原则 – 自学样例

- 例子: 循环

- 矩阵数据: 32×32 , 主存块大小: 8 data $\rightarrow$ 128 blocks
- Cache容量: 64 data, Cache行大小: 8 data $\rightarrow$ 8 lines
- 数据被存储在矩阵行中, 并且按矩阵列读取

- 直接映射Cache

- 强制性缺失: 每个主存块都有一次, 总共128次
- 容量不足缺失: 只有行 3和行 7, 总共224次
- 冲突缺失: 发生在行 0, 1, 2, 4, 5, 6, 总共672次



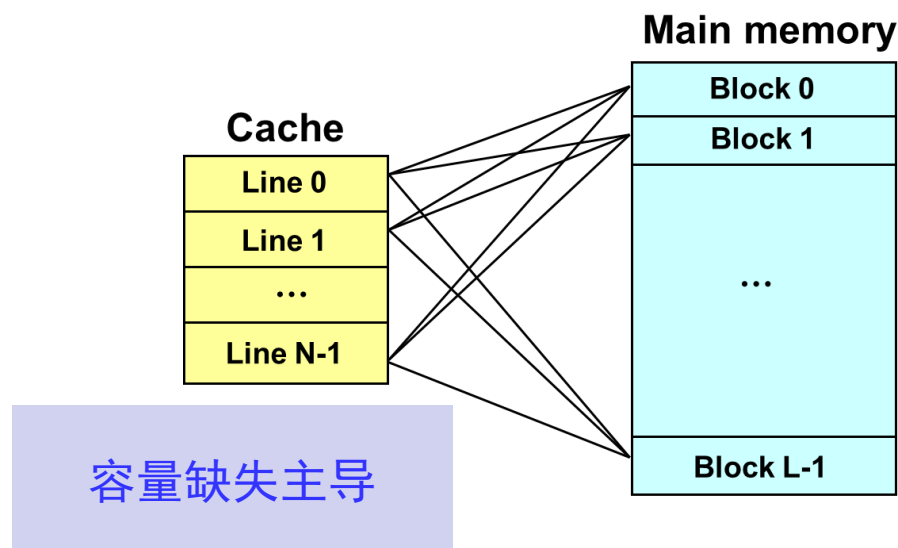
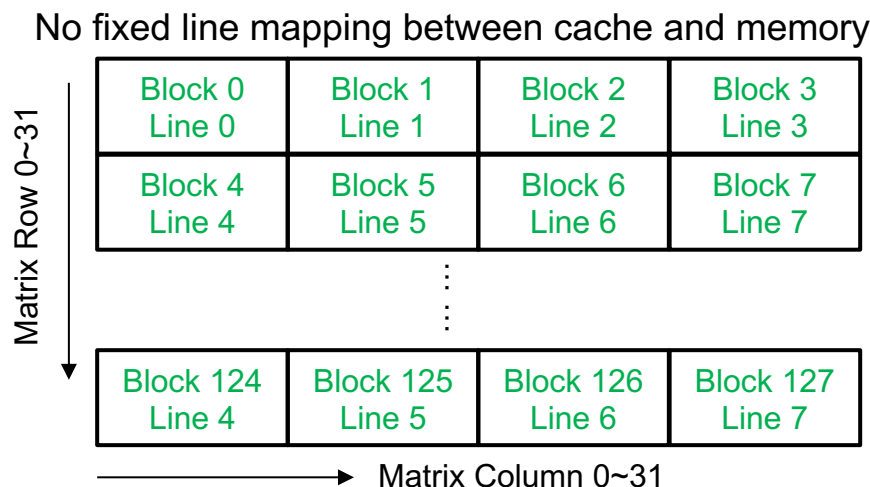
缓存缺失: 3C原则 – 自学样例

- 例子: 循环

- 矩阵数据: 32×32 , 主存块大小: 8 data $\rightarrow$ 128 blocks
- Cache容量: 64 data, Cache行大小: 8 data $\rightarrow$ 8 lines
- 数据被存储在矩阵行中, 并且按矩阵列读取

- 全相联Cache

- 强制性缺失: 每个主存块都有一次, 总共128次
- 容量缺失: 除了首次访问外还缺失了, 总共896次
- 冲突缺失: 不存在



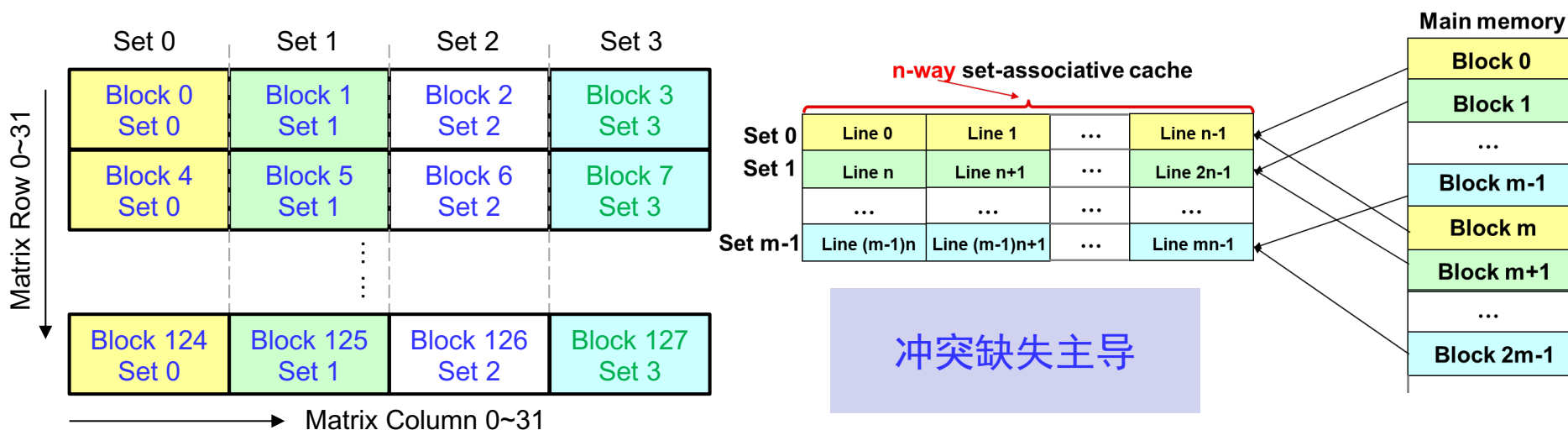
缓存缺失: 3C原则 – 自学样例

• 例子: 循环

- 矩阵数据: 32×32 , 主存块大小: 8 data $\rightarrow$ 128 blocks
- Cache容量: 64 data, Cache行大小: 8 data $\rightarrow$ 8 lines
- 数据被存储在矩阵行中, 并且按矩阵列读取

• 2-路组相联Cache

- 强制性缺失: 每个主存块都有一次, 总共128次
- 容量缺失: 只有组3, 总共224次
- 冲突缺失: 组0, 1, 2中发生, 总共672次



缓存缺失: 3C原则 – 自学样例

- 强制缺失: 每个主存块一次, 总共128次
- 直接映射
 - 容量缺失: 只有行3, 7, 总共224次
 - 冲突缺失: 行0, 1, 2, 4, 5, 6中发生, 总共672次
- 全相联
 - 容量缺失: 除了首次访问外还缺失了, 总共896次
 - 冲突缺失: 不存在
- 2-路组相联
 - 容量缺失: 只有组 3, 总共224次
 - 冲突缺失: 发生在组 0, 1, 2, 总共672次

冲突缺失主导

容量缺失主导

冲突缺失主导

	强制	容量	冲突
直接映射	存在	存在	主导
全相联	存在	主导	/
组相联	存在	存在	主导